

Generative AI Lab Record

Experiment 1

Experiment Title:

Install Matplotlib and display a simple line graph.

Aim:

To install Matplotlib and display a simple line graph.

Code:

```
import matplotlib.pyplot as plt
```

```
x=[1,2,3,4,5]
```

```
y=[2,4,6,8,10]
```

```
plt.plot(x,y,marker="o")
```

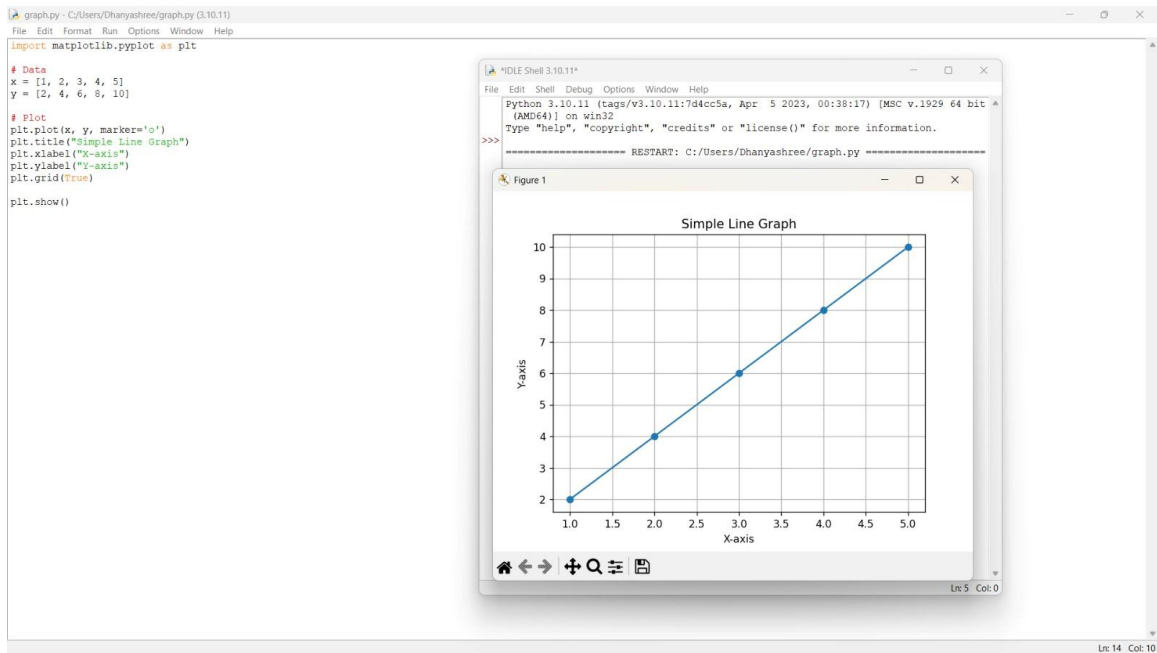
```
plt.title("Line Graph")
```

```
plt.xlabel("X")
```

```
plt.ylabel("Y")
```

```
plt.show()
```

Screenshot:



Result:

Thus, install Matplotlib and display a simple line graph. was completed successfully.

Experiment 2

Experiment Title:

Install Scikit-learn and print its version.

Aim:

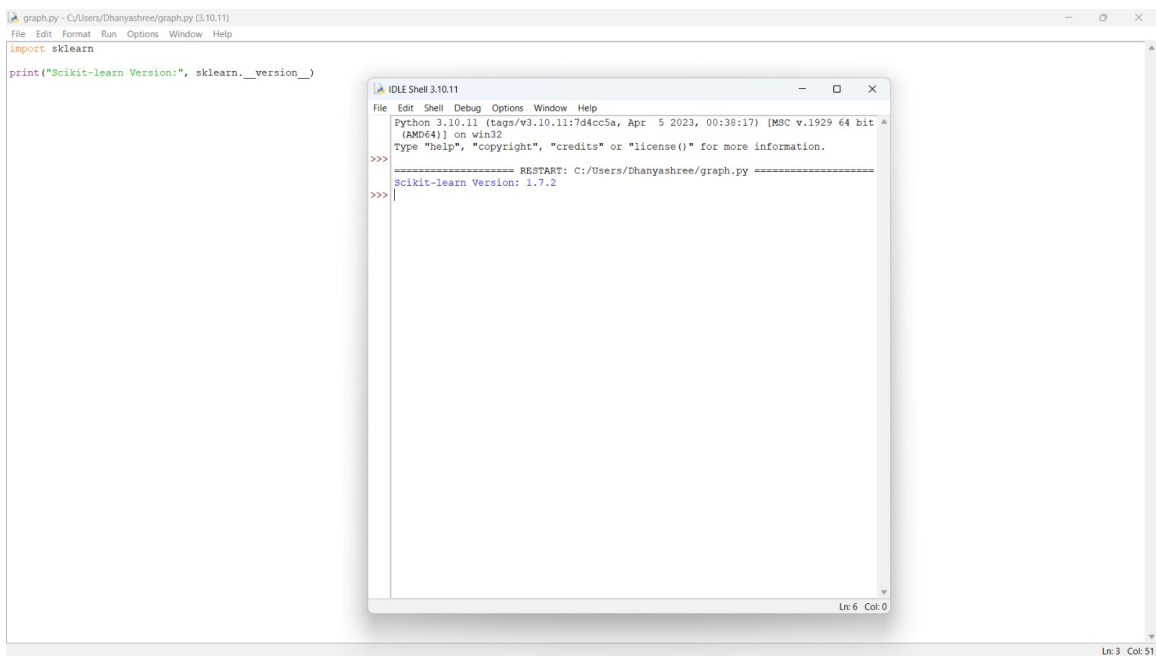
To install Scikit-learn and print its version.

Code:

```
import sklearn
```

```
print(sklearn.__version__)
```

Screenshot:



Result:

Thus, install Scikit-learn and print its version. was completed successfully.

Experiment 3

Experiment Title:

Create a NumPy array containing the first 20 natural numbers.

Aim:

To create a NumPy array containing the first 20 natural numbers.

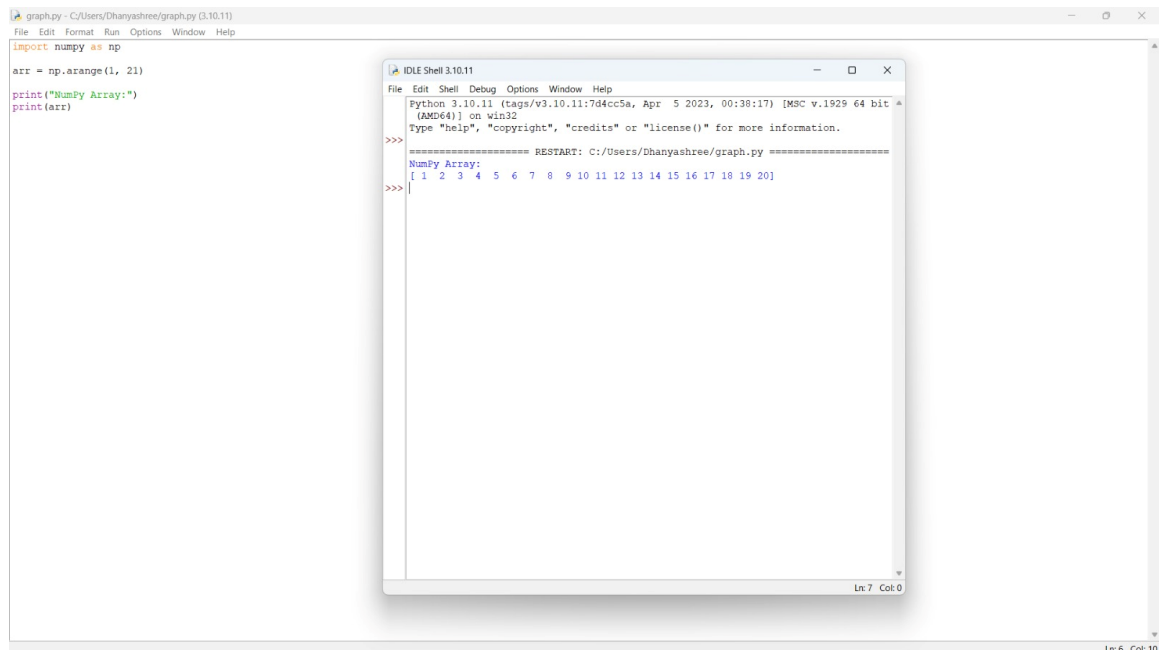
Code:

```
import numpy as np
```

```
a=np.arange(1,21)
```

```
print(a)
```

Screenshot:



Result:

Thus, create a NumPy array containing the first 20 natural numbers. was completed successfully.

Experiment 4

Experiment Title:

Create a Pandas DataFrame with student names and grades.

Aim:

To create a Pandas DataFrame with student names and grades.

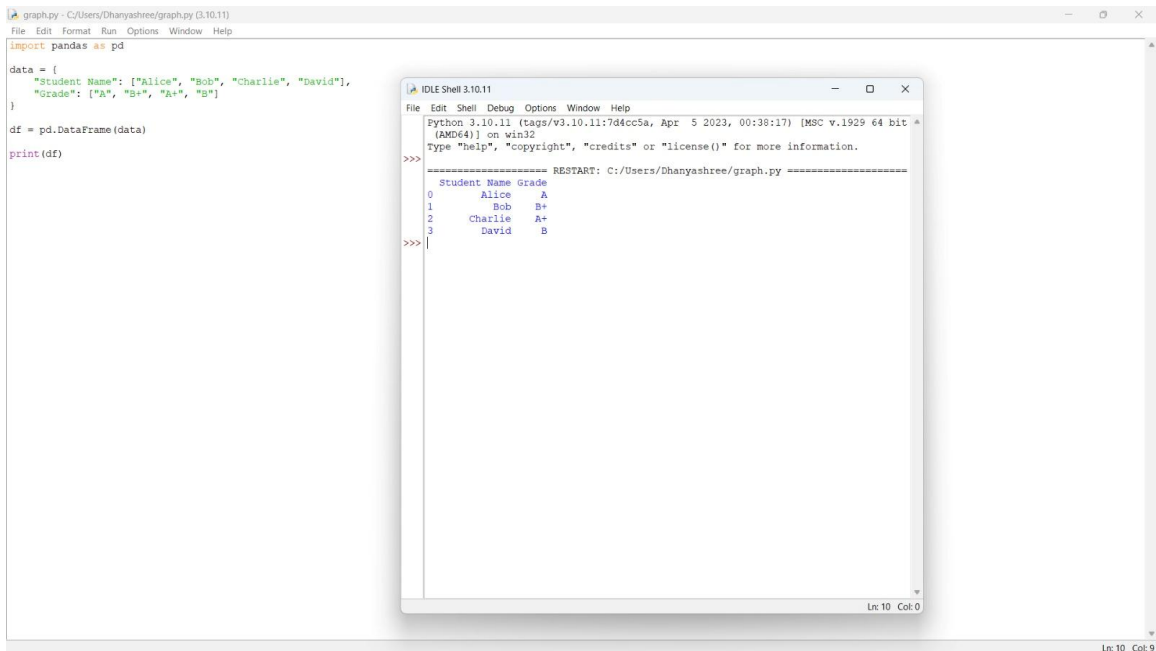
Code:

```
import pandas as pd
```

```
df=pd.DataFrame({  
    "Name":["Ram","Sita","John","Sara"],  
    "Grade":["A","B","A+","A"]  
})
```

```
print(df)
```

Screenshot:



Result:

Thus, create a Pandas DataFrame with student names and grades. was completed successfully.

Experiment 5

Experiment Title:

Verify the installed versions of all AI libraries on your system.

Aim:

To verify the installed versions of all AI libraries on your system.

Code:

```
import numpy

import pandas

import matplotlib

import sklearn

import torch

import transformers


print("NumPy:",numpy.__version__)

print("Pandas:",pandas.__version__)

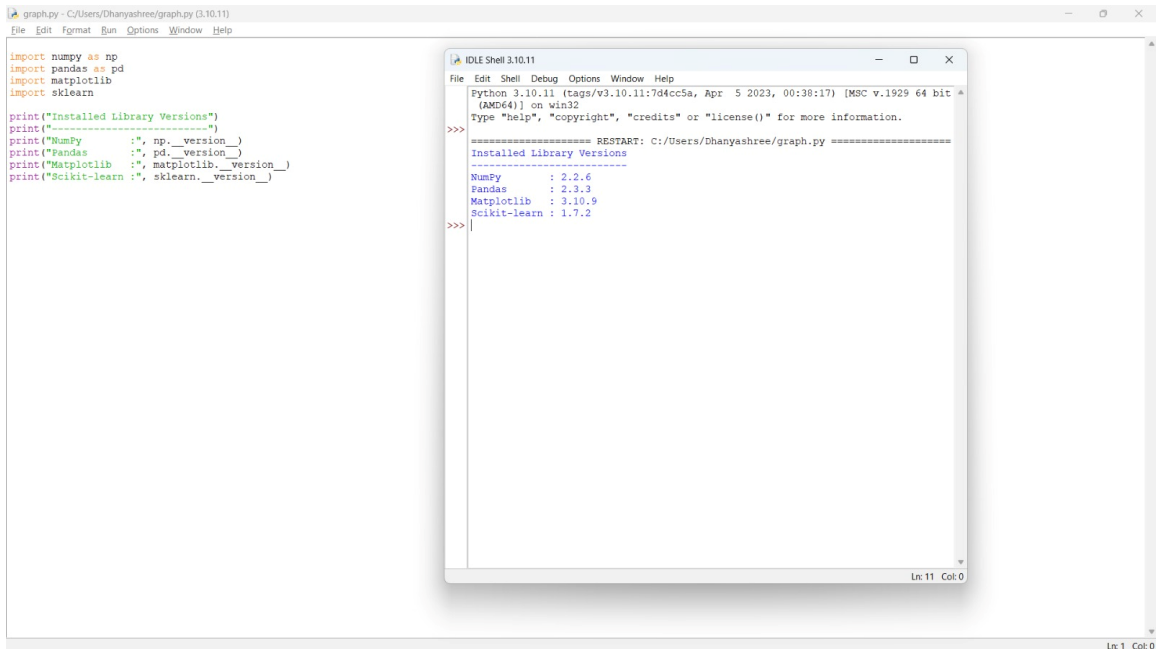
print("Matplotlib:",matplotlib.__version__)

print("Scikit-learn:",sklearn.__version__)

print("PyTorch:",torch.__version__)

print("Transformers:",transformers.__version__)
```

Screenshot:



Result:

Thus, verify the installed versions of all AI libraries on your system. was completed successfully.

Experiment 6

Experiment Title:

Create tensors using PyTorch and perform basic mathematical operations.

Aim:

To create tensors using PyTorch and perform basic mathematical operations.

Code:

```
import torch
```

```
a=torch.tensor([1,2,3])
```

```
b=torch.tensor([4,5,6])
```

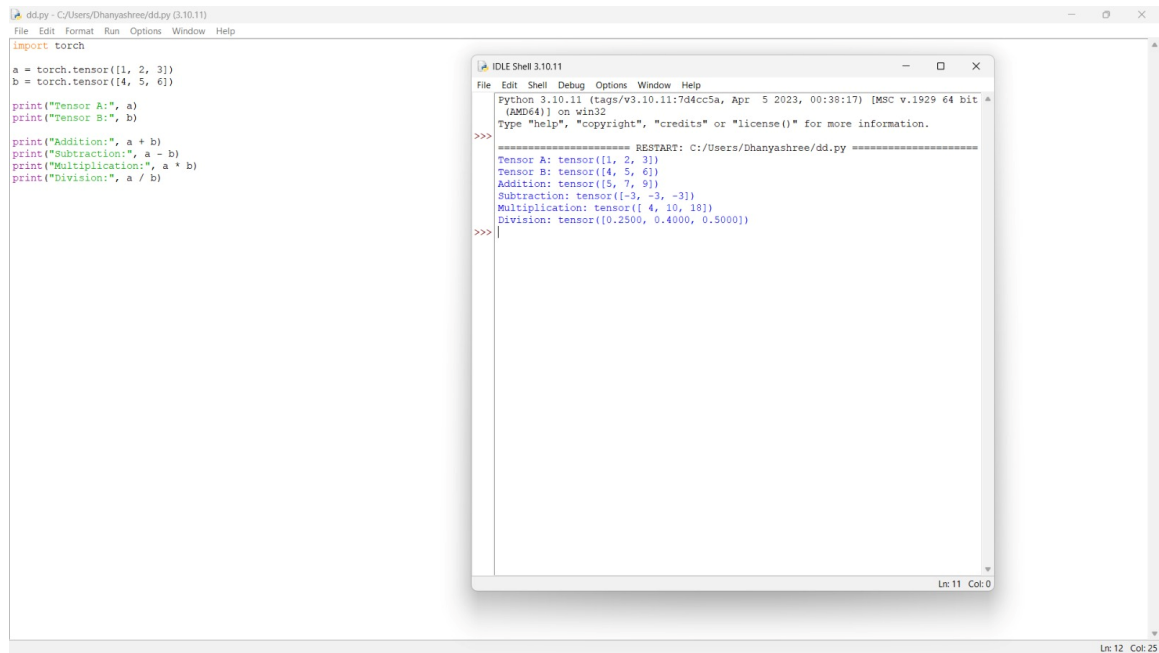
```
print("Tensor A:",a)
```

```
print("Tensor B:",b)
```

```
print("Addition:",a+b)
```

```
print("Multiplication:",a*b)
```

Screenshot:



The screenshot shows a Python IDE window titled 'dd.py - C:/Users/Dhanyashree/dd.py (3.10.11)'. The code in the editor defines two tensors, 'a' and 'b', and performs addition, subtraction, multiplication, and division. An 'IDLE Shell 3.10.11' window is open, displaying the output of the code. The output shows the tensors and the results of the operations: addition, subtraction, multiplication, and division.

```
import torch

a = torch.tensor([1, 2, 3])
b = torch.tensor([4, 5, 6])

print("Tensor A:", a)
print("Tensor B:", b)

print("Addition:", a + b)
print("Subtraction:", a - b)
print("Multiplication:", a * b)
print("Division:", a / b)
```

```
>>>
===== RESTART: C:/Users/Dhanyashree/dd.py =====
Tensor A: tensor([1, 2, 3])
Tensor B: tensor([4, 5, 6])
Addition: tensor([5, 7, 9])
Subtraction: tensor([-3, -3, -3])
Multiplication: tensor([ 4, 10, 18])
Division: tensor([0.2500, 0.4000, 0.5000])
>>>
```

Result:

Thus, create tensors using PyTorch and perform basic mathematical operations. was completed successfully.

Experiment 7

Experiment Title:

Load a Pre-trained Transformer Model using Hugging Face and perform sentiment analysis.

Aim:

To load a Pre-trained Transformer Model using Hugging Face and perform sentiment analysis.

Code:

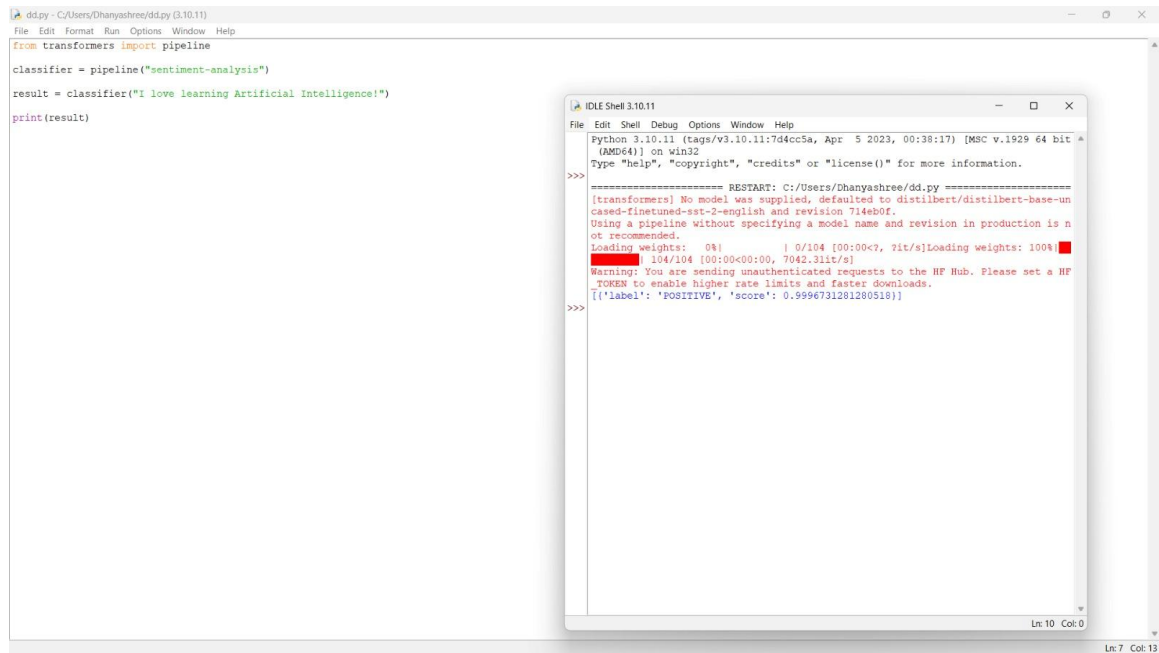
```
from transformers import pipeline
```

```
classifier=pipeline("sentiment-analysis")
```

```
result=classifier("I love Generative AI.")
```

```
print(result)
```


Screenshot:



Result:

Thus, load a Pre-trained Transformer Model using Hugging Face and perform sentiment analysis. was completed successfully.

Experiment 8

Experiment Title:

Tokenization using BERT Tokenizer.

Aim:

To tokenization using BERT Tokenizer.

Code:

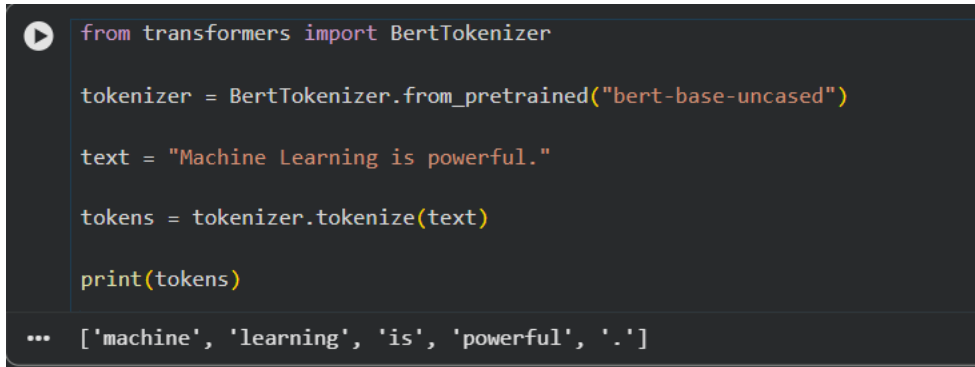
```
from transformers import BertTokenizer
```

```
tokenizer=BertTokenizer.from_pretrained("bert-base-uncased")
```

```
text="Artificial Intelligence is amazing."
```

```
tokens=tokenizer.tokenize(text)
```

```
print(tokens)
```

Screenshot:

```
from transformers import BertTokenizer

tokenizer = BertTokenizer.from_pretrained("bert-base-uncased")

text = "Machine Learning is powerful."

tokens = tokenizer.tokenize(text)

print(tokens)

... ['machine', 'learning', 'is', 'powerful', '.']
```

Result:

Thus, tokenization using BERT Tokenizer. was completed successfully.

Experiment 9**Experiment Title:**

Generate text using a pre-trained GPT-2 model.

Aim:

To generate text using a pre-trained GPT-2 model.

Code:

```
from transformers import pipeline
```

```
generator=pipeline("text-generation",model="gpt2")
```

```
result=generator(

    "Artificial Intelligence",

    max_new_tokens=30,

    do_sample=True

)
```

```
print(result[0]["generated_text"])
```

Screenshot:

```
from transformers import AutoTokenizer, AutoModelForCausalLM

tokenizer = AutoTokenizer.from_pretrained("gpt2")
model = AutoModelForCausalLM.from_pretrained("gpt2")

text = "Artificial Intelligence"

inputs = tokenizer(text, return_tensors="pt")

output = model.generate(
    **inputs,
    max_new_tokens=20,
    do_sample=False,
    pad_token_id=tokenizer.eos_token_id
)

print(tokenizer.decode(output[0], skip_special_tokens=True))
```

... Loading weights: 100%  148/148 [00:00<00:00, 3340.42it/s]

Artificial Intelligence (AI) is a new field of research that has been gaining momentum in recent years.

Result:

Thus, generate text using a pre-trained GPT-2 model. was completed successfully.

Experiment 10

Experiment Title:

Load a pre-trained transformer model using the Hugging Face pipeline() API and perform sentiment analysis.

Aim:

To load a pre-trained transformer model using the Hugging Face pipeline() API and perform sentiment analysis.

Code:

```
from transformers import pipeline
```

```
classifier=pipeline("sentiment-analysis")
```

```
print(classifier("This course is very interesting."))
```

Screenshot:

```
from transformers import pipeline

classifier = pipeline("sentiment-analysis")

result = classifier("I love learning Generative AI.")

print(result)
```

... Loading weights: 100%  104/104 [00:00<00:00, 2354.75it/s]
[{'label': 'POSITIVE', 'score': 0.9996098875999451}]

Result:

Thus, load a pre-trained transformer model using the Hugging Face pipeline() API and perform sentiment analysis. was completed successfully.

Experiment 11

Experiment Title:

Load the BERT tokenizer and tokenize a sentence.

Aim:

To load the BERT tokenizer and tokenize a sentence.

Code:

```
from transformers import BertTokenizer
```

```
tokenizer=BertTokenizer.from_pretrained("bert-base-uncased")
```

```
sentence="Machine Learning is powerful."
```

```
print(tokenizer.tokenize(sentence))
```

Screenshot:

```
from transformers import BertTokenizer

tokenizer = BertTokenizer.from_pretrained("bert-base-uncased")

sentence = "Machine Learning is powerful."

print(tokenizer.tokenize(sentence))
```

... ['machine', 'learning', 'is', 'powerful', '.']

Result:

Thus, load the BERT tokenizer and tokenize a sentence. was completed successfully.

Experiment 12**Experiment Title:**

Load a pre-trained BERT model and generate contextual embeddings.

Aim:

To load a pre-trained BERT model and generate contextual embeddings.

Code:

```
from transformers import BertTokenizer,BertModel
```

```
tokenizer=BertTokenizer.from_pretrained("bert-base-uncased")
```

```
model=BertModel.from_pretrained("bert-base-uncased")
```

```
inputs=tokenizer("Artificial Intelligence",return_tensors="pt")
```

```
outputs=model(**inputs)
```

```
print(outputs.last_hidden_state.shape)
```

Screenshot:

```
from transformers import BertTokenizer, BertModel

tokenizer = BertTokenizer.from_pretrained("bert-base-uncased")
model = BertModel.from_pretrained("bert-base-uncased")

inputs = tokenizer("Artificial Intelligence", return_tensors="pt")

outputs = model(**inputs)

print(outputs.last_hidden_state.shape)
```

... Loading weights: 100% 199/199 [00:00<00:00, 4337.76it/s]
torch.Size([1, 4, 768])

Result: Thus, load a pre-trained BERT model and generate contextual embeddings. was completed successfully.

Experiment 13

Experiment Title:

Generate text using a pre-trained GPT-2 model.

Aim:

To generate text using a pre-trained GPT-2 model.

Code:

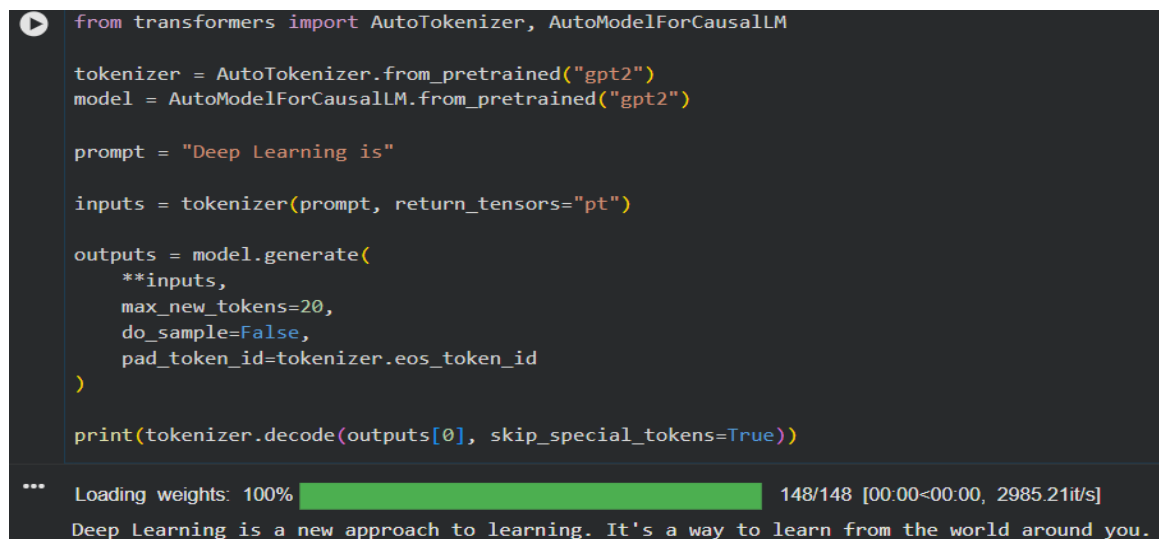
```
from transformers import pipeline
```

```
generator=pipeline("text-generation",model="gpt2")
```

```
result=generator(  
    "Deep Learning",  
    max_new_tokens=40  
)
```

```
print(result[0]["generated_text"])
```

Screenshot:



```
from transformers import AutoTokenizer, AutoModelForCausalLM  
  
tokenizer = AutoTokenizer.from_pretrained("gpt2")  
model = AutoModelForCausalLM.from_pretrained("gpt2")  
  
prompt = "Deep Learning is"  
  
inputs = tokenizer(prompt, return_tensors="pt")  
  
outputs = model.generate(  
    **inputs,  
    max_new_tokens=20,  
    do_sample=False,  
    pad_token_id=tokenizer.eos_token_id  
)  
  
print(tokenizer.decode(outputs[0], skip_special_tokens=True))
```

... Loading weights: 100% 148/148 [00:00<00:00, 2985.21it/s]
Deep Learning is a new approach to learning. It's a way to learn from the world around you.

Result:

Thus, generate text using a pre-trained GPT-2 model. was completed successfully.

Experiment 14

Experiment Title:

Question Answering using a Pre-trained Transformer.

Aim:

To question Answering using a Pre-trained Transformer.

Code:

```
from transformers import AutoTokenizer, AutoModelForQuestionAnswering
```

```
import torch
```

```
model_name = "distilbert-base-cased-distilled-squad"
```

```
tokenizer = AutoTokenizer.from_pretrained(model_name)
```

```
model = AutoModelForQuestionAnswering.from_pretrained(model_name)
```

```
context = "Python was created by Guido van Rossum."
```

```
question = "Who created Python?"
```

```
inputs = tokenizer(question, context, return_tensors="pt")
```

```
with torch.no_grad():
```

```
    outputs = model(**inputs)
```

```
start = torch.argmax(outputs.start_logits)
```

```
end = torch.argmax(outputs.end_logits) + 1
```

```
answer = tokenizer.convert_tokens_to_string(
```

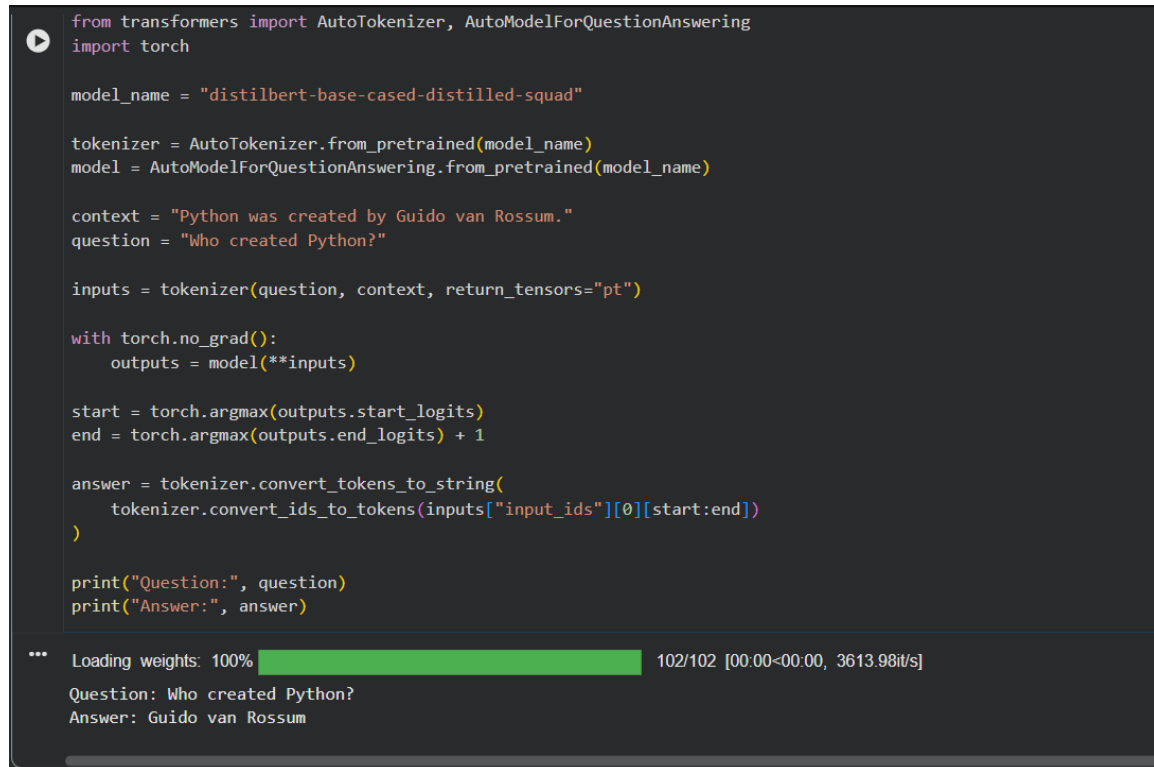
```
    tokenizer.convert_ids_to_tokens(inputs["input_ids"][0][start:end])
```

)

```
print("Question:", question)
```

```
print("Answer:", answer)
```

Screenshot:



```
from transformers import AutoTokenizer, AutoModelForQuestionAnswering
import torch

model_name = "distilbert-base-cased-distilled-squad"

tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModelForQuestionAnswering.from_pretrained(model_name)

context = "Python was created by Guido van Rossum."
question = "Who created Python?"

inputs = tokenizer(question, context, return_tensors="pt")

with torch.no_grad():
    outputs = model(**inputs)

start = torch.argmax(outputs.start_logits)
end = torch.argmax(outputs.end_logits) + 1

answer = tokenizer.convert_tokens_to_string(
    tokenizer.convert_ids_to_tokens(inputs["input_ids"][0][start:end])
)

print("Question:", question)
print("Answer:", answer)
```

... Loading weights: 100% 102/102 [00:00<00:00, 3613.98it/s]

Question: Who created Python?

Answer: Guido van Rossum

Result:

Thus, question Answering using a Pre-trained Transformer. was completed successfully.

Experiment 15

Experiment Title:

Tokenize a sentence using the BERT tokenizer and display the generated tokens and token IDs.

Aim:

To tokenize a sentence using the BERT tokenizer and display the generated tokens and token IDs.

Code:

```
from transformers import BertTokenizer
```



```
tokenizer=BertTokenizer.from_pretrained("bert-base-uncased")
```

```
text="Artificial Intelligence"
```

```
tokens=tokenizer.tokenize(text)
```

```
ids=tokenizer.convert_tokens_to_ids(tokens)
```

```
print("Tokens:",tokens)
```

```
print("Token IDs:",ids)
```

Screenshot:

```
from transformers import BertTokenizer

tokenizer = BertTokenizer.from_pretrained("bert-base-uncased")

text = "Artificial Intelligence"

tokens = tokenizer.tokenize(text)
token_ids = tokenizer.convert_tokens_to_ids(tokens)

print("Tokens :", tokens)
print("Token IDs :", token_ids)
```

```
... Tokens : ['artificial', 'intelligence']
Token IDs : [7976, 4454]
```

Result:

Thus, tokenize a sentence using the BERT tokenizer and display the generated tokens and token IDs. was completed successfully.

Experiment 16

Experiment Title:

Compare the tokenization outputs of BERT and GPT-2.

Aim:

To compare the tokenization outputs of BERT and GPT-2.

Code:

```
from transformers import BertTokenizer,GPT2Tokenizer

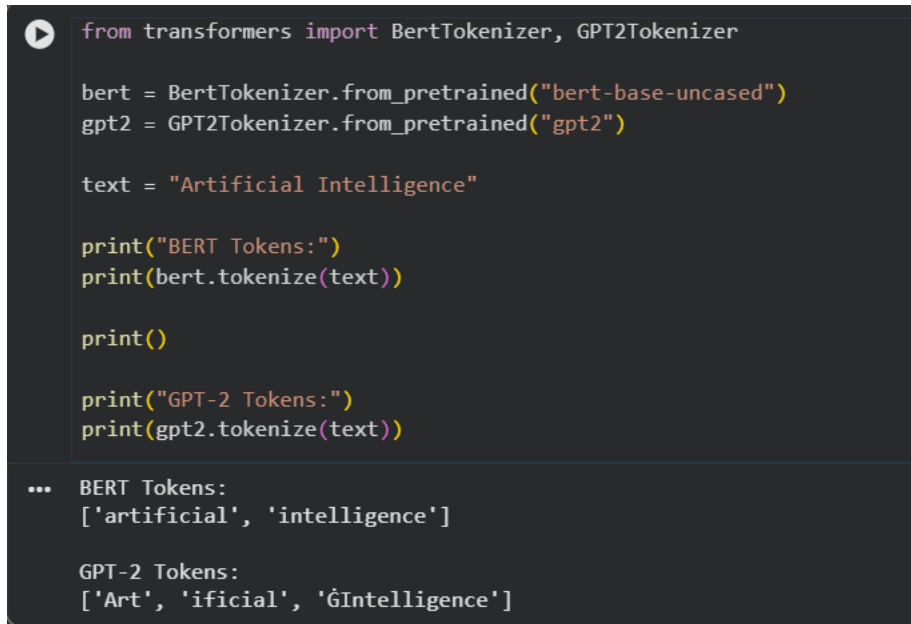
bert=BertTokenizer.from_pretrained("bert-base-uncased")

gpt2=GPT2Tokenizer.from_pretrained("gpt2")

text="Artificial Intelligence is amazing"

print("BERT:",bert.tokenize(text))

print("GPT2:",gpt2.tokenize(text))
```

Screenshot:

```
from transformers import BertTokenizer, GPT2Tokenizer

bert = BertTokenizer.from_pretrained("bert-base-uncased")
gpt2 = GPT2Tokenizer.from_pretrained("gpt2")

text = "Artificial Intelligence"

print("BERT Tokens:")
print(bert.tokenize(text))

print()

print("GPT-2 Tokens:")
print(gpt2.tokenize(text))

... BERT Tokens:
['artificial', 'intelligence']

GPT-2 Tokens:
['Art', 'ificial', 'ĠIntelligence']
```

Result:

Thus, compare the tokenization outputs of BERT and GPT-2. was completed successfully.

Experiment 17**Experiment Title:**

Generate contextual embeddings for a sentence using BERT.

Aim:

To generate contextual embeddings for a sentence using BERT.

Code:

```
from transformers import BertTokenizer,BertModel
```

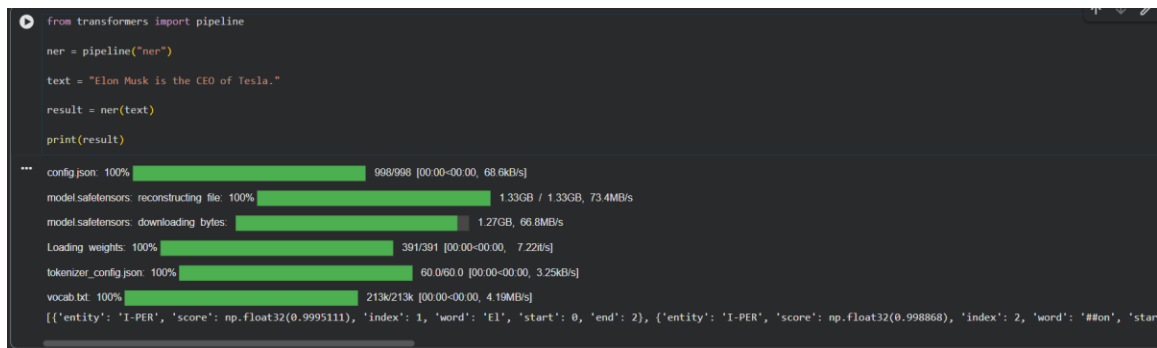
```
tokenizer=BertTokenizer.from_pretrained("bert-base-uncased")
```

```
model=BertModel.from_pretrained("bert-base-uncased")
```

```
inputs=tokenizer("Natural Language Processing",return_tensors="pt")
```

```
outputs=model(**inputs)
```

```
print(outputs.last_hidden_state)
```

Screenshot:

```
from transformers import pipeline

ner = pipeline("ner")

text = "Elon Musk is the CEO of Tesla."

result = ner(text)

print(result)
```

... config.json: 100% ██████████ 998/998 [00:00<00:00, 68.6kB/s]

model.safetensors: reconstructing file: 100% ██████████ 1.33GB / 1.33GB, 73.4MB/s

model.safetensors: downloading bytes: ██████████ 1.27GB, 66.8MB/s

Loading weights: 100% ██████████ 391/391 [00:00<00:00, 7.22kB/s]

tokenizer_config.json: 100% ██████████ 60.0kB/0 [00:00<00:00, 3.25kB/s]

vocab.txt: 100% ██████████ 213k/213k [00:00<00:00, 4.19MB/s]

```
[{'entity': 'I-PER', 'score': np.float32(0.9995111), 'index': 1, 'word': 'El', 'start': 0, 'end': 2}, {'entity': 'I-PER', 'score': np.float32(0.998868), 'index': 2, 'word': '##on', 'start': 2, 'end': 3}, {'entity': 'I-PER', 'score': np.float32(0.9995111), 'index': 3, 'word': 'Musk', 'start': 3, 'end': 7}, {'entity': 'I-PER', 'score': np.float32(0.9995111), 'index': 4, 'word': 'is', 'start': 7, 'end': 8}, {'entity': 'I-PER', 'score': np.float32(0.9995111), 'index': 5, 'word': 'the', 'start': 8, 'end': 9}, {'entity': 'I-PER', 'score': np.float32(0.9995111), 'index': 6, 'word': 'CEO', 'start': 9, 'end': 12}, {'entity': 'I-PER', 'score': np.float32(0.9995111), 'index': 7, 'word': 'of', 'start': 12, 'end': 13}, {'entity': 'I-PER', 'score': np.float32(0.9995111), 'index': 8, 'word': 'Tesla', 'start': 13, 'end': 17}, {'entity': 'I-PER', 'score': np.float32(0.9995111), 'index': 9, 'word': '.', 'start': 17, 'end': 18}]]
```

Result:

Thus, generate contextual embeddings for a sentence using BERT. was completed successfully.

Experiment 18**Experiment Title:**

Compare contextual embeddings generated for two semantically similar sentences.

Aim:

To compare contextual embeddings generated for two semantically similar sentences.

Code:

```
from transformers import BertTokenizer,BertModel

import torch

tokenizer=BertTokenizer.from_pretrained("bert-base-uncased")

model=BertModel.from_pretrained("bert-base-uncased")

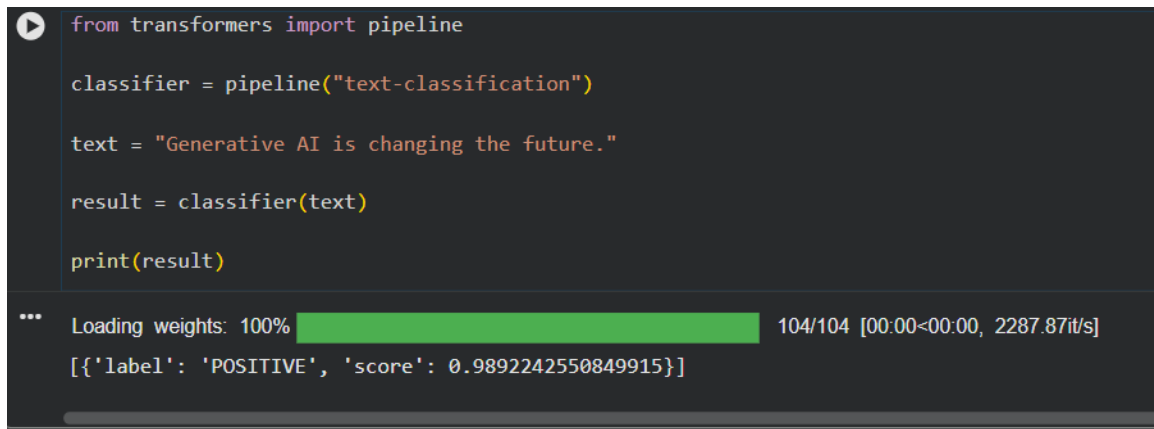
s1="The cat is sleeping."

s2="A cat is taking a nap."

e1=model(**tokenizer(s1,return_tensors="pt")).last_hidden_state.mean(dim=1)

e2=model(**tokenizer(s2,return_tensors="pt")).last_hidden_state.mean(dim=1)

print(torch.cosine_similarity(e1,e2))
```

Screenshot:

```
from transformers import pipeline

classifier = pipeline("text-classification")

text = "Generative AI is changing the future."

result = classifier(text)

print(result)
```

... Loading weights: 100% 104/104 [00:00<00:00, 2287.87it/s]

[{'label': 'POSITIVE', 'score': 0.9892242550849915}]

Result:

Thus, compare contextual embeddings generated for two semantically similar sentences. was completed successfully

Experiment 19

Experiment Title:

Generate text from a prompt using the GPT-2 language model.

Aim:

To generate text from a prompt using the GPT-2 language model.

Code:

```
from transformers import pipeline

generator=pipeline("text-generation",model="gpt2")

result=generator(

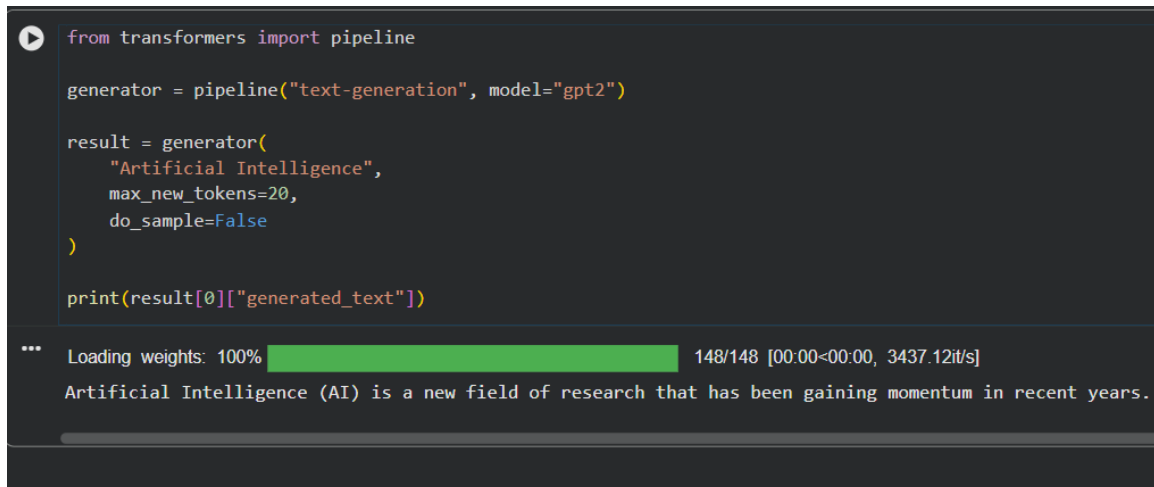
    "The future of Artificial Intelligence",

    max_new_tokens=50

)

print(result[0]["generated_text"])
```

Screenshot:



```
from transformers import pipeline

generator = pipeline("text-generation", model="gpt2")

result = generator(
    "Artificial Intelligence",
    max_new_tokens=20,
    do_sample=False
)

print(result[0]["generated_text"])
```

*** Loading weights: 100% 148/148 [00:00<00:00, 3437.12it/s]

Artificial Intelligence (AI) is a new field of research that has been gaining momentum in recent years.

Result:

Thus, generate text from a prompt using the GPT-2 language model. was completed successfully.

Experiment 20

Experiment Title:

Generate and compare responses for multiple prompts using a pre-trained LLM.

Aim:

To generate and compare responses for multiple prompts using a pre-trained LLM.

Code:

```
from transformers import pipeline

generator=pipeline("text-generation",model="gpt2")

prompts=[

    "Artificial Intelligence",

    "Machine Learning",

    "Data Science"

]

for prompt in prompts:

    print("\nPrompt:",prompt)

    output=generator(prompt,max_new_tokens=30)

    print(output[0]["generated_text"])
```

Screenshot:

```
from transformers import BertTokenizer, BertModel

tokenizer = BertTokenizer.from_pretrained("bert-base-uncased")
model = BertModel.from_pretrained("bert-base-uncased")

text = "Machine Learning"

inputs = tokenizer(text, return_tensors="pt")

outputs = model(**inputs)

print(outputs.last_hidden_state.shape)
```

*** Loading weights: 100%  199/199 [00:00<00:00, 2944.68it/s] torch.Size([1, 4, 768])

Result:

Thus, generate and compare responses for multiple prompts using a pre-trained LLM. was completed successfully.

unit 2

Experiment 1

Experiment Title:

Design Zero-shot, One-shot, and Few-shot prompts to generate a product description for a Smart Fitness Watch.

Aim:

To design Zero-shot, One-shot, and Few-shot prompts for generating a product description for a Smart Fitness Watch.

Code:

```
print("ZERO-SHOT PROMPT")
print("Generate a product description for a Smart Fitness Watch
highlighting its features, benefits, design, and health-tracking
capabilities.")

print("\nONE-SHOT PROMPT")
print("""
Example:
Product: Wireless Earbuds
Description: Wireless Earbuds provide clear sound, Bluetooth
connectivity, noise cancellation, and long battery life.

Now generate a similar product description for a Smart Fitness Watch.
""")
```

```
print("\nFEW-SHOT PROMPT")
print("""
Example 1:
Product: Smart Water Bottle
Description: A smart water bottle tracks daily water intake and
provides hydration reminders.

Example 2:
Product: Wireless Earbuds
Description: Wireless earbuds provide high-quality sound, Bluetooth
connectivity, and long battery life.

Now generate a product description for a Smart Fitness Watch.
""")
```

Output:


```
fg.py - C:/Users/Dhanyashree/fg.py (3.10.11)
File Edit Format Run Options Window Help
from transformers import pipeline

generator = pipeline("text-generation", model="gpt2")

prompt = """
Write a professional product description for a Smart Fitness Watch highlighting its health tracking, battery life, GPS, and connectivity.
"""

output = generator(prompt, max_length=180)
print(output[0]["generated_text"])

IDLE Shell 3.10.11
File Edit Shell Debug Options Window Help
Python 3.10.11 (tags/v3.10.11:7d4cc5a, Apr 5 2023, 00:38:17) [MSC v.1929 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:/Users/Dhanyashree/fg.py =====
' (ReadTimeoutError: HTTPConnectionPool(host='huggingface.co', port=443): Read timed out. (read timeout=10)'), 'Request ID: d893fa95-93bc-4fbc-9e2e-d9fcd8b69c6')' thrown while requesting HEAD https://huggingface.co/gpt2/resolve/main/generation_config.json
Retrying in is (Retry 1/5).
Device set to use cpu
Truncation was not explicitly activated but 'max_length' is provided a specific value, please use 'truncation=True' to explicitly truncate examples to max length. Defaulting to 'longest_first' truncation strategy. If you encode pairs of sequences (HUB-style) with the tokenizer you can select this strategy more precisely by providing a specific strategy to 'truncation'.
Setting 'pad_token_id' to 'eos_token_id':50256 for open-end generation.
Both 'max_new_tokens' (=256) and 'max_length' (=180) seem to have been set. 'max_new_tokens' will take precedence. Please refer to the documentation for more information. (https://huggingface.co/docs/transformers/main/en/main_classes/text_generation)

Write a professional product description for a Smart Fitness Watch highlighting its health tracking, battery life, GPS, and connectivity.

Write a professional product description for a Smart Fitness Watch highlighting its health tracking, battery life, GPS, and connectivity. Provide a professional service that will help you find the best fit for your situation.

Write a professional product description for a Smart Fitness Watch highlighting its health tracking, battery life, GPS, and connectivity. Provide a professional service that will help you find the best fit for your situation. Provide a smart service that will help you find out how much exercise you need to do per day, and how much exercise you can do in the morning.

Write a professional product description for a Smart Fitness Watch highlighting its health tracking, battery life, GPS, and connectivity. Provide a professional service that will help you find the best fit for your situation. Create a simple, easy-to-use, easy-to-use app for your Smart Fitness Watch to find your fitness goals.

Ln 12 Col 0
```

```
fg.py - C:/Users/Dhanyashree/fg.py (3.10.11)
File Edit Format Run Options Window Help
from transformers import pipeline

generator = pipeline("text-generation", model="gpt2")

prompt = """
Example
Product: Wireless Earbuds
Description:
Experience crystal-clear sound with Bluetooth 5.3 connectivity, noise cancellation, and a sleek, ergonomic design.
Now write a similar product description for a Smart Fitness Watch.
"""

output = generator(prompt, max_length=220)
print(output[0]["generated_text"])

IDLE Shell 3.10.11
File Edit Shell Debug Options Window Help
its health tracking, battery life, GPS, and connectivity.

Write a professional product description for a Smart Fitness Watch highlighting its health tracking, battery life, GPS, and connectivity. Provide a professional service that will help you find the best fit for your situation.

Write a professional product description for a Smart Fitness Watch highlighting its health tracking, battery life, GPS, and connectivity. Provide a professional service that will help you find the best fit for your situation. Provide a smart service that will help you find out how much exercise you need to do per day, and how much exercise you can do in the morning.

Write a professional product description for a Smart Fitness Watch highlighting its health tracking, battery life, GPS, and connectivity. Provide a professional service that will help you find the best fit for your situation. Create a simple, easy-to-use, easy-to-use app for your Smart Fitness Watch to find your fitness goals.

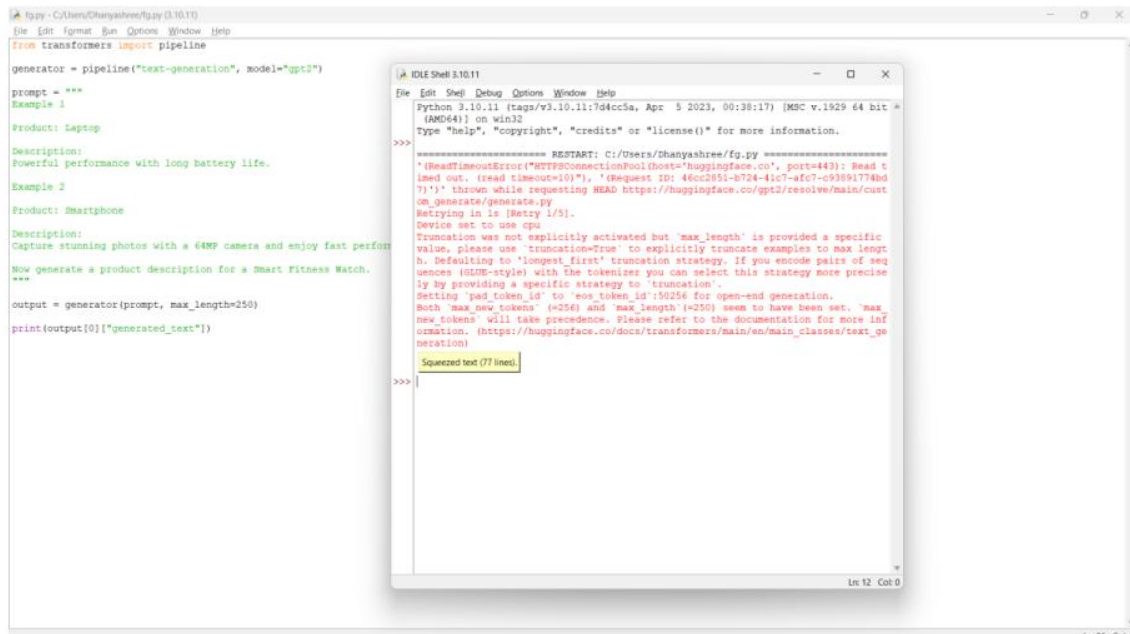
Write a simple, easy-to-use, easy-to-use app for your Smart Fitness Watch to find your fitness goals. Use the FitBit app to find your fitness goals, and even track your progress.

Use the FitBit app to find your fitness goals, and even track your progress. Create a simple, easy-to-use, easy

===== RESTART: C:/Users/Dhanyashree/fg.py =====
Device set to use cpu
Truncation was not explicitly activated but 'max_length' is provided a specific value, please use 'truncation=True' to explicitly truncate examples to max length. Defaulting to 'longest_first' truncation strategy. If you encode pairs of sequences (HUB-style) with the tokenizer you can select this strategy more precisely by providing a specific strategy to 'truncation'.
Setting 'pad_token_id' to 'eos_token_id':50256 for open-end generation.
Both 'max_new_tokens' (=256) and 'max_length' (=220) seem to have been set. 'max_new_tokens' will take precedence. Please refer to the documentation for more information. (https://huggingface.co/docs/transformers/main/en/main_classes/text_generation)

Squeezed text (76 lines)

Ln 18 Col 34
```



Result:

Thus, Zero-shot, One-shot, and Few-shot prompts were successfully designed for generating a Smart Fitness Watch product description.

Experiment 2

Experiment Title:

Design Zero-shot, One-shot, and Few-shot prompts to generate a 200-word blog on Applications of Artificial Intelligence in Healthcare.

Aim:

To design Zero-shot, One-shot, and Few-shot prompts for generating a 200-word blog on Applications of Artificial Intelligence in Healthcare.

Code:

```
print("ZERO-SHOT PROMPT")
print('Write a 200-word blog on "Applications of Artificial Intelligence in Healthcare."')

print("\nONE-SHOT PROMPT")
print("""
Example:
Topic: AI in Education
Blog: Artificial Intelligence improves education through personalized learning,
automated assessment, and intelligent tutoring systems.
```

```
Now write a 200-word blog on "Applications of Artificial Intelligence  
in Healthcare."  
""")
```

```
print("\nFEW-SHOT PROMPT")  
print("""  
Example 1:  
Topic: AI in Education  
Blog: AI supports personalized learning and automated assessment.
```

```
Example 2:  
Topic: AI in Agriculture  
Blog: AI helps farmers monitor crops and improve productivity.
```

```
Now write a 200-word blog on "Applications of Artificial Intelligence  
in Healthcare."  
""")
```

Output:

```
idpy - C:\Users\Chanyshere\idpy (3.10.11)
File Edit Format Run Options Window Help

from transformers import pipeline

generator = pipeline("text-generation", model="gpt2")

# Generate text
prompt = "Write a 200-word blog on Applications of Artificial Intelligence in Healthcare."

result = generator(prompt,
                    max_length=300,
                    do_sample=True,
                    temperature=0.7)

# Print generated blog
print(result[0]["generated_text"])

idpy - C:\Users\Chanyshere\idpy (3.10.11)
File Edit Shell Debug Options Window Help

Device set to use cpu
Truncation was not explicitly activated but 'max_length' is provided a specific
value, please use 'truncation=True' to explicitly truncate examples to max lengt
h. Defaulting to 'longest-first' truncation strategy. If you encode pairs of seq
uences (GLUE-style) with the tokenizer you can select this strategy more precise
ly by providing a specific strategy to 'truncation'.
Setting 'pad_token_id' to 'eos_token_id':50256 for open-end generation.
Both 'max_new_tokens' (256) and 'max_length' (300) seem to have been set. 'max
_new_tokens' will take precedence. Please refer to the documentation for more inf
ormation. (https://huggingface.co/docs/transformers/main/en/main_classes/text_g
eneration)
Write a 200-word blog on Applications of Artificial Intelligence in Healthcare.
Bleatad
Comments (13 posted)
How to install and configure software for personal computing.
What is the most important thing to do in the personal computing world?
How to install software on your home computer.
How to install software on your PC to monitor the health of a patient.
How to install software on your PC to monitor the health of a patient. How to in
stall software on your PC to monitor the health of a patient. How to install sof
tware on your PC to monitor the health of a patient. How to install software on
your PC to monitor the health of a patient. How to install software on your PC t
o monitor the health of a patient. How to install software on your PC to monitor
the health of a patient.
How to install software on your PC to monitor the health of a patient. How to in
stall software on your PC to monitor the health of a patient. How to install sof
tware on your PC to monitor the health of a patient. How to install software on
your PC to monitor the health of a patient. How to install software on your PC t
o monitor the health of a patient. How to install software on your PC to monitor
the health of a patient. How to install software on your PC to monitor the healt
h of a patient. How to install software on your PC to monitor the health of a pa
tient. How to install software on your PC to monitor the health of a patient.
Ln 19 Col 0
```

```
idpy - C:\Users\Chanyshere\idpy (3.10.11)
File Edit Format Run Options Window Help

from transformers import pipeline

generator = pipeline("text-generation", model="gpt2")

prompt = ""
Example:
Topic: Applications of AI in Education
Blog:
Artificial Intelligence is transforming education by enabling personalized learning, au
tomatic grading, and virtual tutors.
Now write a 200-word blog on Applications of Artificial Intelligence in Healthcare.
***
result = generator(prompt, max_length=300)

print(result[0]["generated_text"])

idpy - C:\Users\Chanyshere\idpy (3.10.11)
File Edit Shell Debug Options Window Help

huggingface.co/gpt2/resolve/main/generation_config.json
Retrying in 1s (Retry 1/5).
(MaxRetryError('HTTPSConnectionPool(host='huggingface.co', port=443): Max ret
ries exceeded with url: /gpt2/resolve/main/generation_config.json (Caused by Nam
eResolutionError('HTTPSConnectionPool(host='huggingface.co', port=443): Failed to
resolve 'huggingface.co' ([Errno 11001] getaddrinfo failed)'))', 'Request ID
: e2a27618-6eb4-4126-9516-4a6431da6340'))' thrown while requesting HEAD https://
huggingface.co/gpt2/resolve/main/generation_config.json
Retrying in 2s (Retry 2/5).
Device set to use cpu
Truncation was not explicitly activated but 'max_length' is provided a specific
value, please use 'truncation=True' to explicitly truncate examples to max lengt
h. Defaulting to 'longest-first' truncation strategy. If you encode pairs of seq
uences (GLUE-style) with the tokenizer you can select this strategy more precise
ly by providing a specific strategy to 'truncation'.
Setting 'pad_token_id' to 'eos_token_id':50256 for open-end generation.
Both 'max_new_tokens' (256) and 'max_length' (300) seem to have been set. 'max
_new_tokens' will take precedence. Please refer to the documentation for more inf
ormation. (https://huggingface.co/docs/transformers/main/en/main_classes/text_g
eneration)
Example:
Topic: Applications of AI in Education
Blog:
Artificial Intelligence is transforming education by enabling personalized learn
ing, automated grading, and virtual tutors.
Now write a 200-word blog on Applications of Artificial Intelligence in Healthca
re.
Don't forget to add a disclaimer:
This post is for educational purposes only. It is not intended to be an educatio
nal or political comment, and should not be used to imply endorsement of any spe
cific political or social group or political ideology.
You can contact me with any questions or comments (contact me directly).
Ln 39 Col 0
```

```
from transformers import pipeline

generator = pipeline("text-generation", model="gpt2")

prompt = """
Example 1:
Applications of AI in Education
AI improves personalized learning, grading, and student engagement.

Example 2:
Applications of AI in Agriculture
AI helps monitor crops, predict yields, and optimize irrigation.

Now write a 200-word blog on Applications of Artificial Intelligence in Healthcare.
"""

result = generator(prompt, max_length=320)
print(result[0]["generated_text"])
```

```
Python 3.10.11 (tags/v3.10.11:7d4cc5a, Apr 5 2023, 00:38:17) [MSC v.1929 64 bit
[AMD64]] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:/Users/dhanyashree/dd.py =====
Device set to use gpu
Truncation was not explicitly activated but 'max_length' is provided a specific
value, please use 'truncation=True' to explicitly truncate examples to max lengt
h. Defaulting to 'longest_first' truncation strategy. If you encode pairs of seq
uences (GPU-style) with the tokenizer you can select this strategy more precise
ly by providing a specific strategy to 'truncation'.
Setting 'pad_token_id' to 'eos_token_id' (50256) for open-end generation.
Both 'max_new_tokens' (n256) and 'max_length' (n320) seem to have been set. 'max
_new_tokens' will take precedence. Please refer to the documentation for more inf
ormation. (https://huggingface.co/docs/transformers/main/en/main_classes/text_ge
neration)
Squeezed text (76 lines):
>>>
```

Result:

Thus, Zero-shot, One-shot, and Few-shot prompts were successfully designed for generating a 200-word healthcare AI blog.

Experiment 3

Experiment Title:

Design Zero-shot, One-shot, and Few-shot prompts to summarize an article into 50 words.

Aim:

To design Zero-shot, One-shot, and Few-shot prompts for summarizing an article into 50 words and compare the summaries based on accuracy, completeness, and readability.

Code:

```
article = input("Enter the article: ")

print("\nZERO-SHOT PROMPT")
print("Summarize the following article in exactly 50 words.")
print(article)

print("\nONE-SHOT PROMPT")
print("""
Example:
Article: Artificial Intelligence helps doctors analyze medical images.
Summary: AI helps doctors analyze medical images and improve
healthcare.
```

```
Now summarize the following article in exactly 50 words:
"""
print(article)

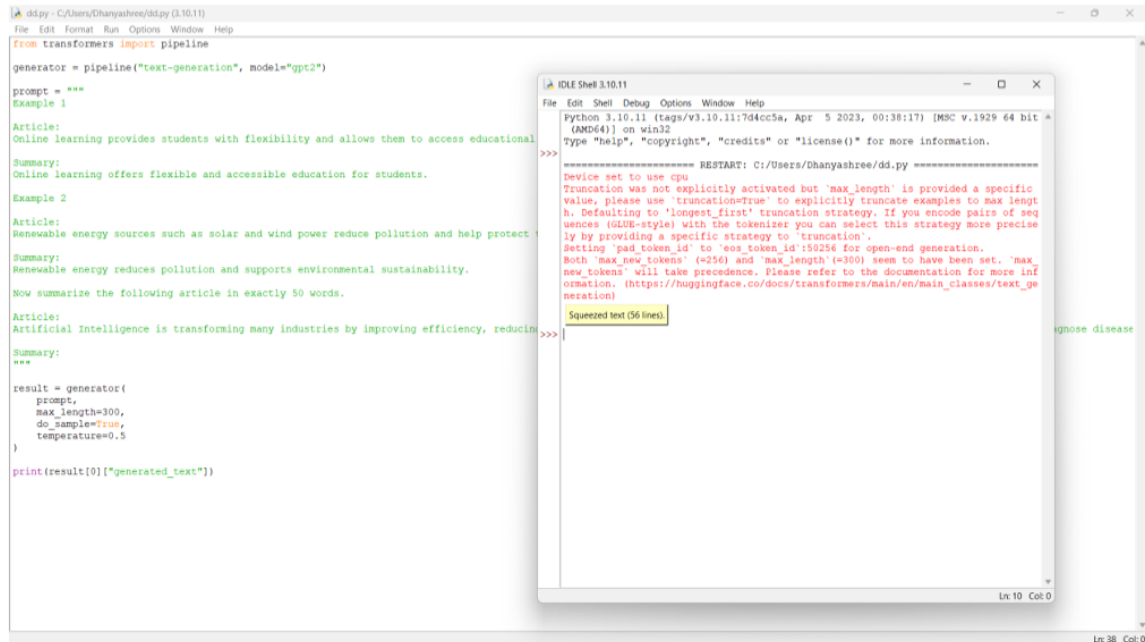
print("\nFEW-SHOT PROMPT")
print("""
Example 1:
Article: Online learning provides flexible education.
Summary: Online learning provides flexible access to education.

Example 2:
Article: Renewable energy reduces pollution.
Summary: Renewable energy supports sustainability by reducing
pollution.

Now summarize the following article in exactly 50 words:
"""
print(article)

print("\nComparison Criteria:")
print("1. Accuracy")
print("2. Completeness")
print("3. Readability")
```

output:



Result:

Thus, Zero-shot, One-shot, and Few-shot prompts were successfully designed for article summarization and comparison. Experiment 4

Experiment Title:

Design Zero-shot, One-shot, and Few-shot prompts to generate a professional email requesting leave due to illness.

Aim:

To design Zero-shot, One-shot, and Few-shot prompts for generating a professional illness leave email and compare them based on tone, grammar, formatting, and completeness.

Code:

```
print("ZERO-SHOT PROMPT")
print("Write a professional email requesting leave due to illness.")
```

```
print("\nNONE-SHOT PROMPT")
print("""
Example:
Subject: Leave Request
```

Dear Sir/Madam,

I request one day of leave due to personal reasons.
I will complete my pending work after returning.

Thank you.

Now write a professional email requesting leave due to illness.
""")

```
print("\nFEW-SHOT PROMPT")  
print("""
```

Example 1:

Subject: Personal Leave

Request leave for one day due to personal reasons.

Example 2:

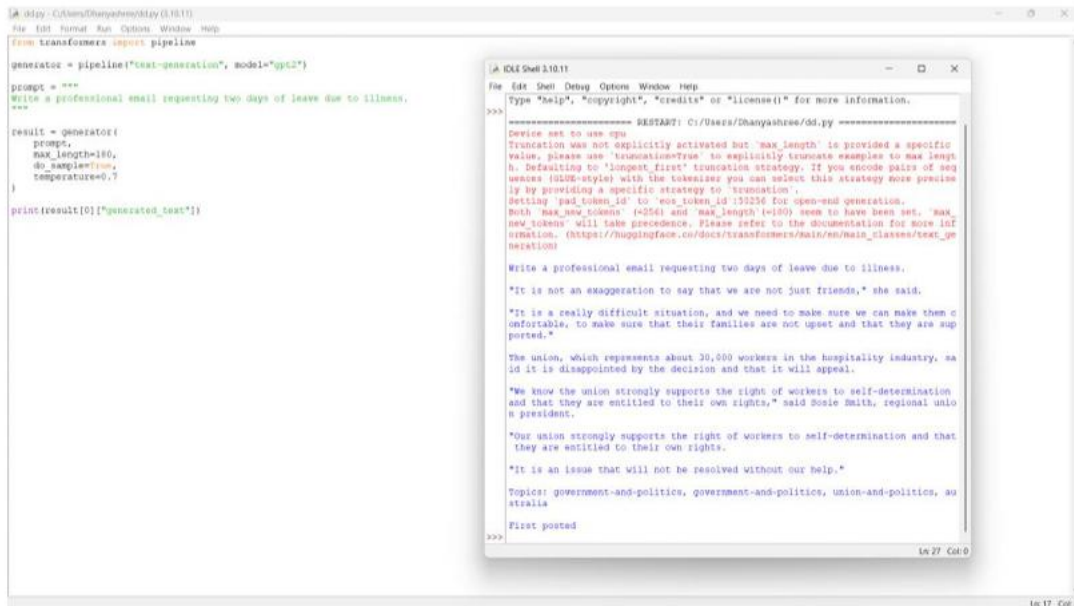
Subject: Medical Leave

Request leave for two days due to illness.

Now write a professional email requesting leave due to illness.
""")

```
print("\nComparison Criteria:")  
print("1. Tone")  
print("2. Grammar")  
print("3. Formatting")  
print("4. Completeness")
```

Output:



The image shows two terminal windows. The left window displays the Python code for a GPT-2 pipeline. The right window shows the output of the pipeline, which includes a restart message, device information, and the generated text for a professional email requesting leave.

```
LA dipy: C:\Users\Dhanyashree\klay (5.18.11)  
File Edit Format Run Options Window Help  
from transformers import pipeline  
  
generator = pipeline("text-generation", model="gpt2")  
  
prompt = ""  
Write a professional email requesting two days of leave due to illness.  
""  
  
result = generator(  
    prompt,  
    max_length=160,  
    do_sample=True,  
    temperature=0.7  
)  
  
print(result[0]["generated_text"])
```

```
LA KKE Shell 2.10.11  
File Edit Shell Debug Options Window Help  
Type "help", "copyright", "credits" or "license()" for more information.  
>>>  
===== RESTART: C:/Users/Dhanyashree/di.py =====  
Device set to use gpu  
Truncation was not explicitly activated but 'max_length' is provided a specific  
value, please use 'truncation=True' to explicitly truncate examples to max length.  
n. Defaulting to 'longest first' truncation strategy. If you encode pairs of sequences (BOS-style) with the tokenizer you can select this strategy more precisely  
by providing a specific strategy to 'truncation'.  
Setting 'pad_token_id' to 'eos_token_id':50256 for open-end generation.  
Both 'max_new_tokens' (4056) and 'max_length' (160) seem to have been set. 'max_  
new_tokens' will take precedence. Please refer to the documentation for more information.  
(https://huggingface.co/docs/transformers/main/en/main_classes/text_generation)  
  
Write a professional email requesting two days of leave due to illness.  
  
"It is not an exaggeration to say that we are not just friends," she said.  
  
"It is a really difficult situation, and we need to make sure we can make them c  
omfortable, to make sure that their families are not upset and that they are sup  
ported."  
  
The union, which represents about 30,000 workers in the hospitality industry, sa  
id it is disappointed by the decision and that it will appeal.  
  
"We know the union strongly supports the right of workers to self-determination  
and that they are entitled to their own rights," said Bonnie Smith, regional unio  
n president.  
  
"Our union strongly supports the right of workers to self-determination and that  
they are entitled to their own rights.  
  
"It is an issue that will not be resolved without our help."  
  
Topic: government-and-politics, government-and-politics, union-and-politics, au  
stralia  
  
First posted  
  
>>>
```

```
ddipy - C:/Users/Dhanyashree/ddipy (3.10.11)
File Edit Format Run Options Window Help
from transformers import pipeline

generator = pipeline("text-generation", model="gpt2")

prompt = """
Example
Subject: Leave Request

Dear Manager,

I request one day of leave due to a family emergency. I will resume work tomorrow.

Thank you.

Regards,
John

Now write a professional email requesting two days of leave due to illness.
"""

result = generator(
    prompt,
    max_length=220,
    do_sample=True,
    temperature=0.7
)

print(result[0]["generated_text"])
```

```
Python 3.10.11
File Edit Shell Debug Options Window Help

I request one day of leave due to a family emergency. I will resume work tomorrow.

Thank you.

Regards,
John

Now write a professional email requesting two days of leave due to illness.

Subject: Leave Request

Dear Manager,

I hope to do a full day's leave. I am extremely worried about getting sick.

I was told that since my family emergency I must leave my job. I am also very worried about getting sick and want to ensure that it is not taken on by another employee.

I have to stay within my work schedule and will not be able to do any work for more than two days. I do not want to leave any other employees alone.

I'm currently experiencing a health crisis. I am not sure how long it will take to get back to work, which is why I am contacting the company right now.

How do I get back to work if I am sick or injured?

I am writing to you and asking that you consider a job offer. If you do not have a job offer, please contact the company directly.

You are the only person who can make me feel safer. Please tell the person you are writing to that you are sick, injured or waiting for them to know I am sick and that I am in need of your help.

I am asking you to contact your supervisor to get an immediate response.

I am currently in

Ln 46 Col 0
```

```
ddipy - C:/Users/Dhanyashree/ddipy (3.10.11)
File Edit Format Run Options Window Help
from transformers import pipeline

generator = pipeline("text-generation", model="gpt2")

prompt = """
Example 1
Subject: Casual Leave

Dear Sir,

I request one day of casual leave due to personal work.

Regards,
Rahul

Example 2
Subject: Medical Leave

Dear Manager,

I am suffering from a fever and request two days of medical leave.

Regards,
Anjali

Now write a professional email requesting two days of leave due to illness.
"""

result = generator(
    prompt,
    max_length=250,
    do_sample=True,
    temperature=0.7
)

print(result[0]["generated_text"])
```

```
Python 3.10.11 (tags/v3.10.11:7d4cc5e, Apr 5 2023, 00:38:17) [MSC v.1929 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.

>>>
Restart: C:/Users/Dhanyashree/ddipy
=====
Device set to use cpu
Truncation was not explicitly activated but 'max_length' is provided a specific value, please use 'truncation=True' to explicitly truncate examples to max length. Defaulting to 'longest_first' truncation strategy. If you encode pairs of sequences (GLUE-style) with the tokenizer you can select this strategy more precisely by providing a specific strategy to 'truncation'.
Setting 'pad_token_id' to 'eos_token_id':50256 for open-end generation.
Both 'max_new_tokens' (=256) and 'max_length' (=250) seem to have been set. 'max_new_tokens' will take precedence. Please refer to the documentation for more information. (https://huggingface.co/docs/transformers/main/en/main_classes/text_generation)
Squeezed text (100 lines)
>>>
```

Result:

Thus, Zero-shot, One-shot, and Few-shot prompts were successfully designed and compared for generating a professional leave email.

Experiment 5

Experiment Title:

Design Zero-shot, One-shot, and Few-shot prompts to generate a promotional social media post for an AI Workshop.

Aim:

To design Zero-shot, One-shot, and Few-shot prompts for generating a promotional social media post for an AI Workshop.

Code:

```
print("ZERO-SHOT PROMPT")
print("Create a promotional social media post for an AI Workshop.")

print("\nONE-SHOT PROMPT")
print("""
Example:
Event: Python Workshop
Post: Join our Python Workshop to improve your programming skills
through
hands-on learning and practical activities. Register today!

Now create a promotional social media post for an AI Workshop.
""")

print("\nFEW-SHOT PROMPT")
print("""
Example 1:
Event: Data Science Workshop
Post: Learn Data Science through practical sessions and real-world
examples.

Example 2:
Event: Cybersecurity Workshop
Post: Learn cybersecurity concepts and protect digital systems with
expert guidance.

Now create a promotional social media post for an AI Workshop.
""")
```

Output:

```
idpy - C:\Users\Chanyashree\idpy (3.10.11)
File Edit Format Run Options Window Help

from transformers import pipeline

generator = pipeline("text-generation", model="gpt2")

prompt = """
Create a promotional social media post for an AI Workshop.
"""

result = generator(
    prompt,
    max_length=150,
    do_sample=True,
    temperature=0.7
)

print(result[0]["generated_text"])
```

```
idpy Shell 3.10.11
File Edit Shell Debug Options Window Help

b. Defaulting to 'longest first' truncation strategy. If you encode pairs of sequences (GLUE-style) with the tokenizer you can select this strategy more precisely by providing a specific strategy to 'truncation'.
Setting 'pad_token_id' to 'eos_token_id':150256 for open-end generation.
Both 'max_new_tokens' (=256) and 'max_length' (=150) seem to have been set. 'max_new_tokens' will take precedence. Please refer to the documentation for more information. (https://huggingface.co/docs/transformers/main/en/main_classes/text_generation)

Create a promotional social media post for an AI Workshop.

Create a referral-based marketing campaign for an AI Workshop.

Increase your exposure to AI training and AI services by spending time on AI training.

Increase your awareness of AI training and AI services by spending time on AI training.

Increase your exposure to AI training and AI services by spending time on AI training.

Advertising or social media campaigns can be done manually or through a database or by using an API. You can also use a website's search engine.

You can also use an API. You can also use a website's search engine.

The following is a list of various options to choose from:

You can also choose to automate the process of selecting courses. To do this you can download the software or go to "Calculating for Training".

You can also download the software or go to "Calculating for Training". You can then select a course that you want to send a link to the AI training service (for example, the AI training website would be the same as the training site you use to view the training). Once you have selected the course you want to send it to the AI training service for your training to be used.

You can then choose a course that you

>>>
```

```
idpy - C:\Users\Chanyashree\idpy (3.10.11)
File Edit Format Run Options Window Help

from transformers import pipeline

generator = pipeline("text-generation", model="gpt2")

prompt = """
Example
Workshop: Cyber Security Workshop
Social Media Post:
Join our Cyber Security Workshop and learn ethical hacking, network security, and cyber defense from industry experts. Register today and boost your career! #CyberSecurity #Workshop
Now create a promotional social media post for an AI Workshop.
"""

result = generator(
    prompt,
    max_length=220,
    do_sample=True,
    temperature=0.7
)

print(result[0]["generated_text"])
```

```
idpy Shell 3.10.11
File Edit Shell Debug Options Window Help

Workshop: Cyber Security Workshop

Social Media Post:
Join our Cyber Security Workshop and learn ethical hacking, network security, and cyber defense from industry experts. Register today and boost your career! #CyberSecurity #Workshop

Now create a promotional social media post for an AI Workshop.

If you have already created a profile for a Workshop, you can still use it in your profile by following the link in the profile.

We will then add your profile to our blog post with the description in the topic "How to Create a Blog Post with the Social Media Marketing Social Network".

If you have created a profile with the Social Media Marketing Social Network, you are invited to participate in our "Tech Talk" at the following workshop: Cyber Security Workshop

Social Media Post:
Join our Cyber Security Workshop and learn ethical hacking, network security, and cyber defense from industry experts. Register today and boost your career! #CyberSecurity #Workshop

Our online workshop is very popular with students from universities, colleges, and even private companies.

Social Media Post:
Join our Cyber security Workshop and learn ethical hacking, network security, and cyber defense from industry experts. Register today and boost your career! #CyberSecurity #Workshop

Now create a promotional social media post for an AI Workshop.

If you have already created a profile for a Workshop, you can still use it in your profile by following the link in the profile.

We will then add your
```

```
idpy - C:\Users\Chanyashree\idpy (3.10.11)
File Edit Format Run Options Window Help

from transformers import pipeline

generator = pipeline("text-generation", model="gpt2")

prompt = """
Example 1
Workshop: Cloud Computing
Prompt:
Join our Cloud Computing Workshop to learn AWS, Azure, and cloud technologies.
Example 2
Workshop: Data Science
Prompt:
Enroll now in our course with hands-on projects in Python, Machine Learning, and Data Science.
Now create a promotional social media post for an AI Workshop.
"""

result = generator(
    prompt,
    max_length=200,
    do_sample=True,
    temperature=0.7
)

print(result[0]["generated_text"])
```

```
idpy Shell 3.10.11
File Edit Shell Debug Options Window Help

Python 3.10.11 (tags/v3.10.11:11f7b55c, Aug 9 2023, 55:16:17) [AMD64] on win32
Type "help()", "copyright()", "credits()" or "license()" for more information.

>>>
===== WARNING: C:\Users\Chanyashree\idpy =====
Please run this file with Python 3.10.11 (tags/v3.10.11:11f7b55c, Aug 9 2023, 55:16:17) [AMD64] on win32
Type "help()", "copyright()", "credits()" or "license()" for more information.

b. Defaulting to 'longest first' truncation strategy. If you encode pairs of sequences (GLUE-style) with the tokenizer you can select this strategy more precisely by providing a specific strategy to 'truncation'.
Setting 'pad_token_id' to 'eos_token_id':150256 for open-end generation.
Both 'max_new_tokens' (=256) and 'max_length' (=200) seem to have been set. 'max_new_tokens' will take precedence. Please refer to the documentation for more information. (https://huggingface.co/docs/transformers/main/en/main_classes/text_generation)

>>>
===== WARNING: C:\Users\Chanyashree\idpy =====
Please run this file with Python 3.10.11 (tags/v3.10.11:11f7b55c, Aug 9 2023, 55:16:17) [AMD64] on win32
Type "help()", "copyright()", "credits()" or "license()" for more information.

b. Defaulting to 'longest first' truncation strategy. If you encode pairs of sequences (GLUE-style) with the tokenizer you can select this strategy more precisely by providing a specific strategy to 'truncation'.
Setting 'pad_token_id' to 'eos_token_id':150256 for open-end generation.
Both 'max_new_tokens' (=256) and 'max_length' (=200) seem to have been set. 'max_new_tokens' will take precedence. Please refer to the documentation for more information. (https://huggingface.co/docs/transformers/main/en/main_classes/text_generation)

>>>
```

Result:

Thus, Zero-shot, One-shot, and Few-shot prompts were successfully designed for generating a promotional social media post for an AI Workshop.

Experiment 7**Experiment Title:**

Text Generation using Google Gemini LLM API

Aim:

To generate text responses using the Google Gemini API for different user prompts.

Code:

```
from google import genai

client = genai.Client(api_key="YOUR_GEMINI_API_KEY")

prompt = input("Enter your prompt: ")

response = client.models.generate_content(
    model="gemini-2.5-flash",
    contents=prompt
)

print("\nGenerated Response:\n")
print(response.text)
```

Result:

Thus, text was successfully generated using the Google Gemini LLM API.

Experiment 8**Experiment Title:**

Text Generation using Hugging Face Inference API

Aim:

To generate text using the Hugging Face Inference API based on a user-provided prompt.

Code:

```
import requests

API_URL = "https://api-inference.huggingface.co/models/gpt2"

headers = {
```

```

        "Authorization": "Bearer YOUR_HUGGINGFACE_API_TOKEN"
    }

prompt = input("Enter your prompt: ")

response = requests.post(
    API_URL,
    headers=headers,
    json={"inputs": prompt}
)

result = response.json()

if isinstance(result, list):
    print(result[0]["generated_text"])
else:
    print(result)

```

Result:

Thus, text was successfully generated using the Hugging Face Inference API.

Experiment 9

Experiment Title:

Python Code Generation using Structured Prompting

Aim:

To design structured prompts that instruct an LLM to generate Python programs for specified computational problems.

Code:

```

problem = input("Enter the programming problem: ")

prompt = f"""
Generate a Python program for the following problem.

Problem:
{problem}

Requirements:
1. Use Python.
2. Add comments.
3. Display the output.
4. Explain the logic.
"""

print("\nStructured Prompt:\n")
print(prompt)

```

output:

```

----- RESTART: E:/GEN AI LAB/UNIT 2 EXP 9.py -----
Enter the programming problem: find factorial of a number

Structured Prompt:

Generate a Python program.

Problem:
find factorial of a number

Requirements:
1. Use Python.
2. Add comments.
3. Display the output.
>

```

Result:

Thus, a structured prompt for Python code generation was successfully designed.

Experiment 10

Experiment Title:

SQL Query Generation using Structured Prompting

Aim:

To design structured prompts that provide database schema, table details, and requirements for generating SQL queries.

Code:

```

table = input("Enter table name: ")
columns = input("Enter column names: ")
task = input("Enter the SQL requirement: ")

prompt = f"""
Database Schema

Table Name:
{table}

Columns:
{columns}

Task:
{task}

```

```
Generate the appropriate SQL query.
"""
```

```
print("\nStructured Prompt:\n")
print(prompt)
```

output:

```
----- RESTART: E:\JEN AI LAB
Enter table name: students
Enter column names: RollNo, Name, Marks
Enter the SQL requirement: Display students whose marks are greater than 80.

Structured Prompt:

Database Schema

Table Name:
students

Columns:
RollNo, Name, Marks

Task:
Display students whose marks are greater than 80.

Generate the appropriate SQL query.
>
```

Result:

Thus, a structured prompt for SQL query generation was successfully designed.

Experiment 11

Experiment Title:

Comparative Analysis of Zero-shot, One-shot and Few-shot Prompting

Aim:

To compare Zero-shot, One-shot, and Few-shot prompting techniques.

Code:

```
task = input("Enter the task: ")

print("\n===== ZERO-SHOT =====")
print(f"Task:\n{task}")

print("\n===== ONE-SHOT =====")
print("""
Example:
Input: Write a Python program to add two numbers.
Output:
a = 10
b = 20
print(a+b)
""")
```



```

"""

print("Now perform:")
print(task)

print("\n===== FEW-SHOT =====")
print("""
Example 1:
Input: Find the sum of two numbers.
Output:
print(a+b)

Example 2:
Input: Find the product of two numbers.
Output:
print(a*b)
""")

print("Now perform:")
print(task)

```

output:

```

Enter the task: Write a Python program to calculate the square of a number.

===== ZERO-SHOT =====
Task:
Write a Python program to calculate the square of a number.

===== ONE-SHOT =====

Example:
Input: Write a Python program to add two numbers.
Output:
a = 10
b = 20
print(a+b)

Now perform:
Write a Python program to calculate the square of a number.

===== FEW-SHOT =====

Example 1:
Input: Find the sum of two numbers.
Output:
print(a+b)

Example 2:
Input: Find the product of two numbers.
Output:
print(a*b)

Now perform:
Write a Python program to calculate the square of a number.
>

```

Result:

Thus, Zero-shot, One-shot, and Few-shot prompting techniques were successfully compared.

Experiment 12

Experiment Title:

Prompt Evaluation, Comparison and Refinement using LLM Responses

Aim:

To evaluate, compare, and refine prompts using predefined quality criteria.

Code:

```
prompt1 = input("Enter Prompt 1: ")
prompt2 = input("Enter Prompt 2: ")

print("\nEvaluation Criteria")
print("1. Relevance")
print("2. Accuracy")
print("3. Completeness")
print("4. Clarity")
print("5. Format Adherence")

choice = input("\nSelect the better prompt (1 or 2): ")

print("\nRefined Prompt:")

if choice == "1":
    print(prompt1)
elif choice == "2":
    print(prompt2)
else:
    print("Invalid choice")
```

output:

Result:

Thus, prompts were successfully evaluated, compared, and the best-performing prompt was refined.

Unit-4

Experiment 1

Experiment Title:

AI Chatbot for Engineering College Student Queries

Aim:

To develop an AI chatbot that answers student queries related to an engineering college using a pre-trained language model.

Code:

```
from transformers import pipeline

chatbot = pipeline("text2text-generation", model="google/flan-t5-small")

question = input("Enter your query: ")

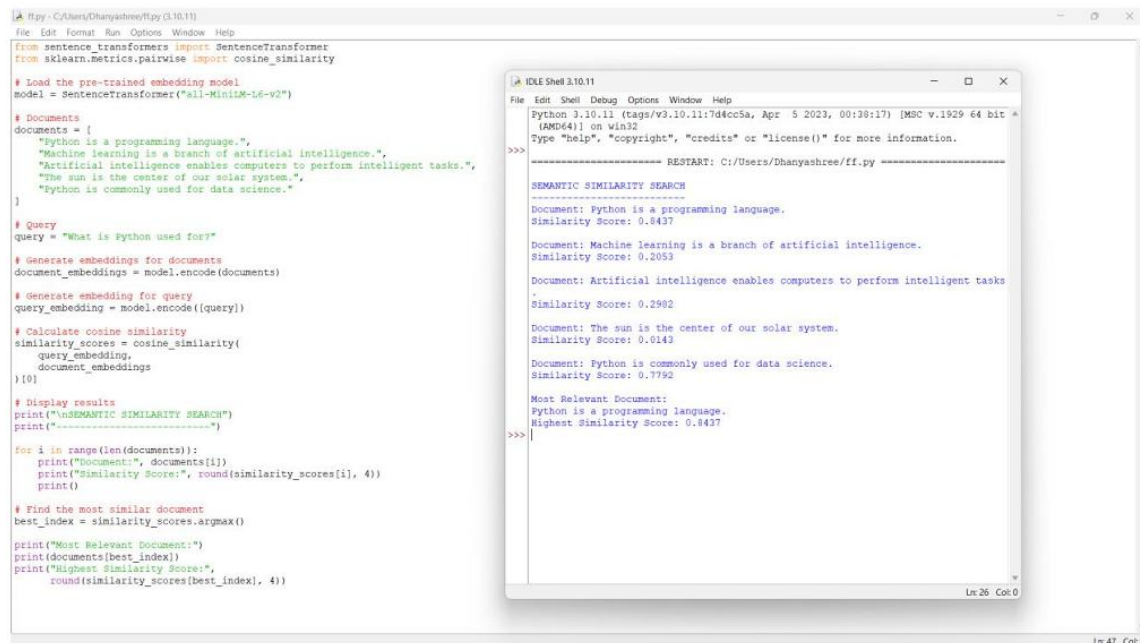
prompt = f"""
You are an engineering college assistant.
Answer the student's question clearly and briefly.

Student question: {question}
"""

result = chatbot(prompt, max_new_tokens=50)

print("\nChatbot Response:")
print(result[0]["generated_text"])
```

Output:

The image shows a screenshot of a Python script in an IDE (left) and its output in a terminal window (right). The script uses the transformers library to create a chatbot pipeline. It defines a list of documents related to Python and machine learning, a query, and calculates cosine similarity between the query and the documents. The output window shows the results of the semantic similarity search, listing documents and their similarity scores, and identifying the most relevant document as 'Python is a programming language.' with a similarity score of 0.8437.

```
File Edit Format Run Options Window Help
from sentence_transformers import SentenceTransformer
from sklearn.metrics.pairwise import cosine_similarity

# Load the pre-trained embedding model
model = SentenceTransformer("all-MiniLM-L6-v2")

# Documents
documents = [
    "Python is a programming language.",
    "Machine learning is a branch of artificial intelligence.",
    "Artificial intelligence enables computers to perform intelligent tasks.",
    "The sun is the center of our solar system.",
    "Python is commonly used for data science."
]

# Query
query = "What is Python used for?"

# Generate embeddings for documents
document_embeddings = model.encode(documents)

# Generate embedding for query
query_embedding = model.encode([query])

# Calculate cosine similarity
similarity_scores = cosine_similarity(
    query_embedding,
    document_embeddings
)[0]

# Display results
print("\nSEMANTIC SIMILARITY SEARCH")
print("-----")

for i in range(len(documents)):
    print("Document:", documents[i])
    print("Similarity Score:", round(similarity_scores[i], 4))
    print()

# Find the most similar document
best_index = similarity_scores.argmax()

print("Most Relevant Document:")
print(documents[best_index])
print("Highest Similarity Score:",
      round(similarity_scores[best_index], 4))

Python 3.10.11 (tags/v3.10.11:7d4cc5a, Apr 5 2023, 00:38:17) [MSC v.1929 64 bit
(AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
RESTART: C:/Users/Dhanyashree/ff.py

SEMANTIC SIMILARITY SEARCH
-----
Document: Python is a programming language.
Similarity Score: 0.8437

Document: Machine learning is a branch of artificial intelligence.
Similarity Score: 0.2053

Document: Artificial intelligence enables computers to perform intelligent tasks
Similarity Score: 0.2982

Document: The sun is the center of our solar system.
Similarity Score: 0.0143

Document: Python is commonly used for data science.
Similarity Score: 0.7792

Most Relevant Document:
Python is a programming language.
Highest Similarity Score: 0.8437
>>>
```

Result:

Thus, an AI chatbot was successfully developed to answer engineering college student queries.

Experiment 2

Experiment Title:

Engineering-Support Chatbot using NLP

Aim:

To develop an engineering-support chatbot that answers technical questions using NLP techniques.

Code:

```
from transformers import pipeline

model = pipeline("text2text-generation", model="google/flan-t5-small")

question = input("Enter your technical question: ")

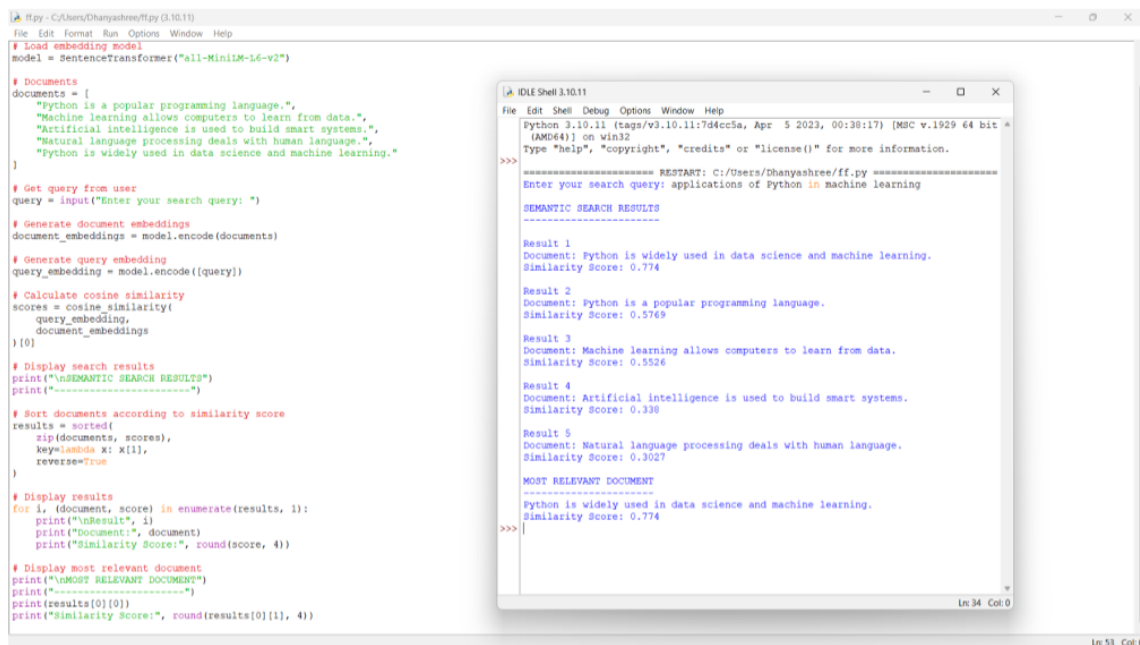
prompt = f"""
You are an engineering technical assistant.
Provide a simple and useful solution to the following question.

Question: {question}
"""

result = model(prompt, max_new_tokens=60)

print("\nTechnical Solution:")
print(result[0]["generated_text"])
```

Output:

The image shows a screenshot of a Python script in an IDE (left) and its output in a terminal window (right). The script is a semantic search tool that uses a SentenceTransformer model to find relevant documents based on a user query. The output displays the top 5 search results with their similarity scores and identifies the most relevant document.

```
ffpy - C:/Users/Dhanyashree/ffpy (3.10.11)
File Edit Format Run Options Window Help
# Load embedding model
model = SentenceTransformer("all-MiniLM-L6-v2")

# Documents
documents = [
    "Python is a popular programming language.",
    "Machine learning allows computers to learn from data.",
    "Artificial intelligence is used to build smart systems.",
    "Natural language processing deals with human language.",
    "Python is widely used in data science and machine learning."
]

# Get query from user
query = input("Enter your search query: ")

# Generate document embeddings
document_embeddings = model.encode(documents)

# Generate query embedding
query_embedding = model.encode([query])

# Calculate cosine similarity
scores = cosine_similarity(
    query_embedding,
    document_embeddings
)[0]

# Display search results
print("\nSEMANTIC SEARCH RESULTS")
print("-----")

# Sort documents according to similarity score
results = sorted(
    zip(documents, scores),
    key=lambda x: x[1],
    reverse=True
)

# Display results
for i, (document, score) in enumerate(results, 1):
    print(f"Result {i}")
    print(f"Document: {document}")
    print(f"Similarity Score: {round(score, 4)}")

# Display most relevant document
print("\nMOST RELEVANT DOCUMENT")
print("-----")
print(results[0][0])
print(f"Similarity Score: {round(results[0][1], 4)}")

Python 3.10.11 (tags/v3.10.11:7d4cc5a, Apr 5 2023, 00:38:17) [MSC v.1929 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.

>>>
===== RESTART: C:/Users/Dhanyashree/ff.py =====
Enter your search query: applications of Python in machine learning

SEMANTIC SEARCH RESULTS
-----

Result 1
Document: Python is widely used in data science and machine learning.
Similarity Score: 0.774

Result 2
Document: Python is a popular programming language.
Similarity Score: 0.5769

Result 3
Document: Machine learning allows computers to learn from data.
Similarity Score: 0.5526

Result 4
Document: Artificial intelligence is used to build smart systems.
Similarity Score: 0.338

Result 5
Document: Natural language processing deals with human language.
Similarity Score: 0.3027

MOST RELEVANT DOCUMENT
-----
Python is widely used in data science and machine learning.
Similarity Score: 0.774
>>>
```

Result:

Thus, an engineering-support chatbot was successfully developed using NLP.

Experiment 3**Experiment Title:**

Engineering Image Generation using Text-to-Image Model

Aim:

To generate an engineering-related image from a suitable text prompt using a pre-trained text-to-image model.

Code:

```
from diffusers import StableDiffusionPipeline
import torch

model_id = "runwayml/stable-diffusion-v1-5"

pipe = StableDiffusionPipeline.from_pretrained(
    model_id,
    torch_dtype=torch.float32
)

pipe = pipe.to("cpu")

prompt = "A modern suspension bridge over a river, engineering design, realistic"

image = pipe(prompt, num_inference_steps=10).images[0]

display(image)
```

Output:

```

# Machine learning allows computers to learn from data.
# Artificial intelligence is used to build smart systems.
# Natural language processing deals with human language.
# Python is widely used for data science.
# Deep learning uses neural networks to learn complex patterns.
# Computer networks allow computers to communicate with each other.
]

# Generate document embeddings
embeddings = model.encode(documents)

# Convert embeddings to NumPy array
embeddings = np.array(embeddings).astype("float32")

# Get embedding dimension
dimension = embeddings.shape[1]

# Create FAISS index
index = faiss.IndexFlatL2(dimension)

# Add document embeddings to FAISS
index.add(embeddings)

print("Number of documents stored in FAISS:", index.ntotal)

# Get query from user
query = input("\nEnter your search query: ")

# Generate query embedding
query_embedding = model.encode([query])
query_embedding = np.array(query_embedding).astype("float32")

# Number of results to retrieve
k = 3

# Search FAISS database
distances, indices = index.search(query_embedding, k)

# Display results
print("\nTop", k, "SIMILAR DOCUMENTS")
print("-----")

for i in range(k):
    document_index = indices[0][i]

    print("\nResult", i + 1)
    print("Document:", documents[document_index])
    print("Distance:", round(float(distances[0][i]), 4))

print("\nSearch completed successfully.")

```

```

Python 3.10.11 (tags/v3.10.11:704cc5a, Apr 5 2023, 00:38:17) [MSC v.1929 64 bit
(AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:/Users/Dhanyashree/ff.py =====
Number of documents stored in FAISS: 7

Enter your search query: Python for machine learning

TOP 3 SIMILAR DOCUMENTS
-----

Result 1
Document: Python is widely used for data science.
Distance: 0.631

Result 2
Document: Python is a popular programming language.
Distance: 0.7763

Result 3
Document: Machine learning allows computers to learn from data.
Distance: 1.085

Search completed successfully.
>>>

```

Result:

Thus, an engineering-related image was successfully generated from a text prompt.

This experiment can take longer on Colab CPU. If GPU is available, select **Runtime** → **Change runtime type** → **T4 GPU**.

Experiment 4

Experiment Title:

Comparison of Images Generated from Different Text Prompts

Aim:

To generate images from different text prompts and compare the effect of prompt changes on the generated images.

Code:

```

from diffusers import StableDiffusionPipeline
import torch

model_id = "runwayml/stable-diffusion-v1-5"

pipe = StableDiffusionPipeline.from_pretrained(
    model_id,
    torch_dtype=torch.float32
)

pipe = pipe.to("cpu")

```

```

prompts = [
    "A modern robotic arm in an engineering laboratory",
    "A futuristic robotic arm in an advanced engineering laboratory",
    "A small industrial robotic arm working in a factory"
]

for prompt in prompts:
    image = pipe(prompt, num_inference_steps=10).images[0]

    print("\nPrompt:", prompt)
    display(image)

```

Output:

The screenshot shows a Python IDE with a script on the left and its execution output on the right. The script, located at `C:/Users/Dhanyashree/ff.py`, performs the following steps:

- Converts document embeddings to NumPy float32.
- Gets the vector dimension (8).
- Creates a FAISS vector database.
- Stores document embeddings in the database.
- Prints the total documents stored (8).
- Gets a query from the user: "Python for data science".
- Converts the query into an embedding.
- Asks the user for the top-k value (3).
- Searches the vector database for the top-3 results.
- Displays the retrieved documents.

The execution output shows the following results:

```

===== RESTART: C:/Users/Dhanyashree/ff.py =====
DOCUMENT VECTOR DATABASE
Total documents stored: 8
Enter your query: Python for data science
Enter number of documents to retrieve (K): 3
TOP-K RETRIEVED DOCUMENTS
Rank: 1
Document: Python is commonly used for data science.
Distance: 0.2768
Rank: 2
Document: Python is a high level programming language.
Distance: 0.719
Rank: 3
Document: Database management systems store and manage data.
Distance: 1.3548
Retrieval completed successfully.

```

Result:

Thus, multiple images were generated successfully, and changes in text prompts produced different visual results.

Experiment 5

Experiment Title:

Speech-to-Text Application for Engineering Queries

Aim:

To convert a user's spoken engineering-related query into written text using a pre-trained AI model.

Code:

```
!pip install -q openai-whisper

import whisper

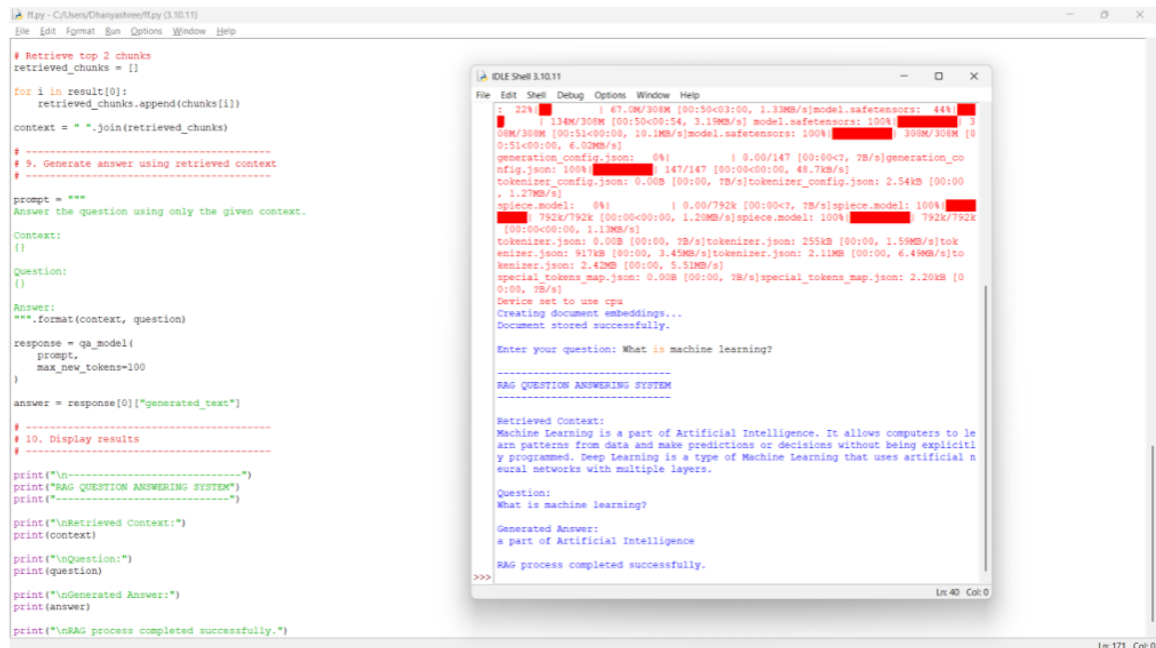
model = whisper.load_model("base")

audio_file = input("Enter audio file path: ")

result = model.transcribe(audio_file)

print("\nTranscribed Text:")
print(result["text"])
```

Output:



```
# Retrieve top 2 chunks
retrieved_chunks = []
for i in result[0]:
    retrieved_chunks.append(chunks[i])
context = " ".join(retrieved_chunks)

# 9. Generate answer using retrieved context
prompt = """
Answer the question using only the given context.
Context:
{}
Question:
{}
Answer:
""".format(context, question)

response = qa_model(
    prompt,
    max_new_tokens=100
)
answer = response[0]["generated_text"]

# 10. Display results
print("\nRetrieved Context:")
print(context)
print("\nQuestion:")
print(question)
print("\nGenerated Answer:")
print(answer)
print("\nRAG process completed successfully.")
```

```
22% 134M/308M [00:50<03:00, 1.33MB/s]model.safetensors: 44% 134M/308M [00:50<03:00, 1.33MB/s]model.safetensors: 100% 134M/308M [00:51<00:00, 10.1MB/s]model.safetensors: 100% 134M/308M [00:51<00:00, 6.02MB/s]
generation_config.json: 0% 0.00/147 [00:00<00:00, 0.00KB/s]generation_config.json: 100% 147/147 [00:00<00:00, 48.7KB/s]
tokenizer_config.json: 0.00B [00:00, 7B/s]tokenizer_config.json: 2.54KB [00:00, 1.27MB/s]
piece.model: 0% 0.00/792k [00:00<00:00, 1.33MB/s]piece.model: 100% 792k/792k [00:00<00:00, 1.33MB/s]
tokenizer.json: 0.00B [00:00, 7B/s]tokenizer.json: 255kB [00:00, 1.59MB/s]tokenizer.json: 917kB [00:00, 3.45MB/s]tokenizer.json: 2.11MB [00:00, 6.49MB/s]tokenizer.json: 2.42MB [00:00, 5.51MB/s]
special_tokens_map.json: 0.00B [00:00, 7B/s]special_tokens_map.json: 2.20kB [00:00, 7B/s]
Device set to use cpu
Creating document embeddings...
Document stored successfully.

Enter your question: What is machine learning?

RAG QUESTION ANSWERING SYSTEM

Retrieved Context:
Machine Learning is a part of Artificial Intelligence. It allows computers to learn patterns from data and make predictions or decisions without being explicitly programmed. Deep Learning is a type of Machine Learning that uses artificial neural networks with multiple layers.

Question:
What is machine learning?

Generated Answer:
a part of Artificial Intelligence

RAG process completed successfully.
```

Result:

Thus, the spoken engineering query was successfully converted into text.

Experiment 6

Experiment Title:

Text-to-Speech Application for Engineering Text

Aim:

To convert engineering-related text into natural-sounding speech using a pre-trained AI model.

Code:

```
from gtts import gTTS
from IPython.display import Audio, display

text = input("Enter engineering text: ")

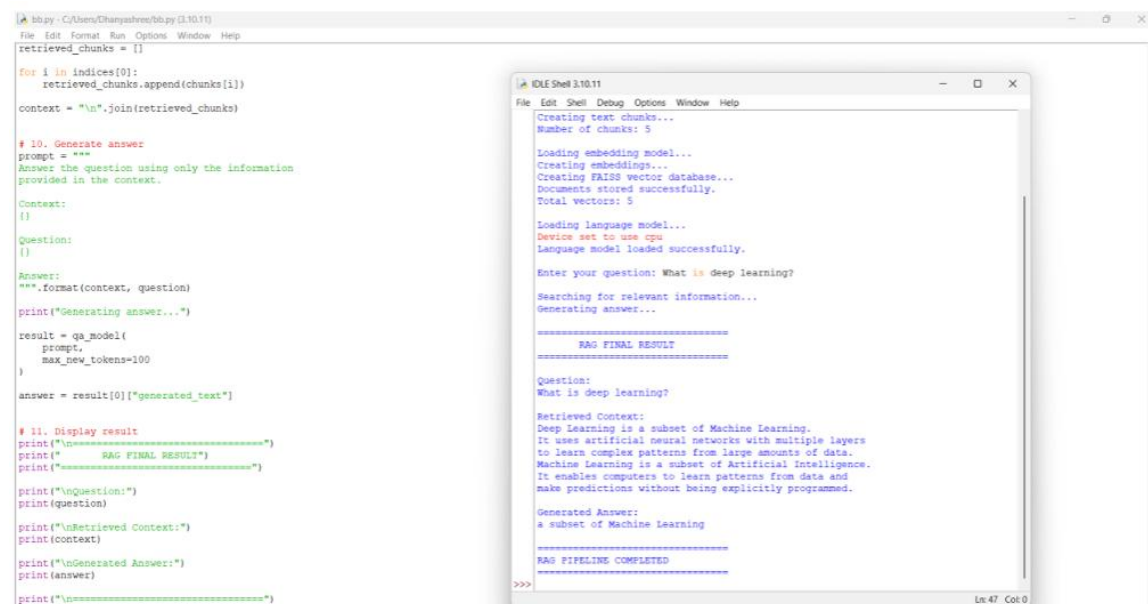
speech = gTTS(text=text, lang="en")

speech.save("engineering_audio.mp3")

print("Speech generated successfully.")

display(Audio("engineering_audio.mp3"))
```

Output:



The image shows two overlapping windows from a code editor. The background window displays a Python script for generating speech from text. The foreground window shows the execution output of a Retrieval-Augmented Generation (RAG) pipeline.

```
bbpy - C:/Users/Changshree/bb.py (3.10.11)
File Edit Format Run Options Window Help

retrieved_chunks = []
for i in indices[0]:
    retrieved_chunks.append(chunks[i])
context = "\n".join(retrieved_chunks)

# 10. Generate answer
prompt = """
Answer the question using only the information
provided in the context.

Context:
{}

Question:
{}

Answer:
""".format(context, question)
print("Generating answer...")

result = qa_model(
    prompt,
    max_new_tokens=100
)

answer = result[0]["generated_text"]

# 11. Display result
print("\n=====")
print("      RAG FINAL RESULT")
print("=====")
print("\nQuestion:")
print(question)

print("\nRetrieved Context:")
print(context)

print("\nGenerated Answer:")
print(answer)
print("\n=====")
```

```
IDE Shell 3.10.11
File Edit Shell Debug Options Window Help

Creating text chunks...
Number of chunks: 5

Loading embedding model...
Creating embeddings...
Creating FAISS vector database...
Documents stored successfully.
Total vectors: 5

Loading language model...
Device set to use gpu
Language model loaded successfully.

Enter your question: What is deep learning?
Searching for relevant information...
Generating answer...

=====
RAG FINAL RESULT
=====

Question:
What is deep learning?

Retrieved Context:
Deep Learning is a subset of Machine Learning.
It uses artificial neural networks with multiple layers
to learn complex patterns from large amounts of data.
Machine Learning is a subset of Artificial Intelligence.
It enables computers to learn patterns from data and
make predictions without being explicitly programmed.

Generated Answer:
a subset of Machine Learning

=====
RAG PIPELINE COMPLETED
=====
>>>
```

Result:

Thus, engineering-related text was successfully converted into speech.

Experiment 7

Experiment Title:

Engineering Document Summarization

Aim:

To summarize a lengthy engineering document into a short and meaningful summary using a pre-trained language model.

Code:

```

from transformers import pipeline

summarizer = pipeline(
    "summarization",
    model="facebook/bart-large-cnn"
)

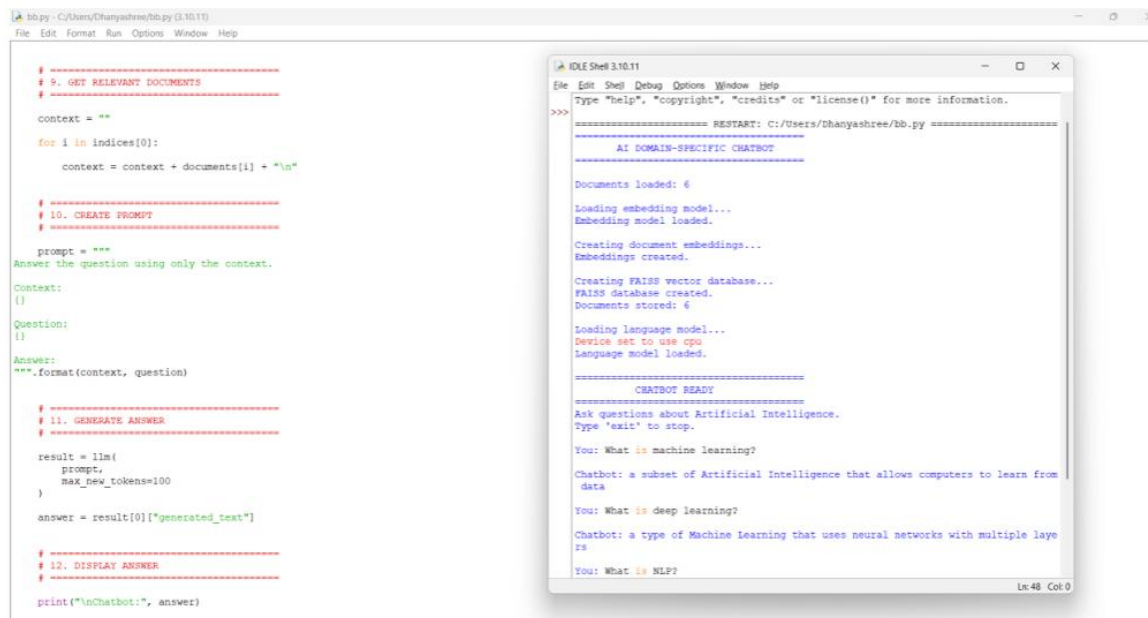
text = input("Enter the document text: ")

result = summarizer(
    text,
    max_length=50,
    min_length=20,
    do_sample=False
)

print("\nSummary:")
print(result[0]["summary_text"])

```

Output:



The left screenshot shows a Python script in a text editor. The script defines a summarizer using the BART-large-CNN model and takes user input to generate a summary. The right screenshot shows the terminal output of the script, which includes the following text:

```

===== RESTART: C:/Users/Dhanyashree/bb.py =====
AI DOMAIN-SPECIFIC CHATBOT

Documents loaded: 6
Loading embedding model...
Embedding model loaded.
Creating document embeddings...
Embeddings created.
Creating FAISS vector database...
FAISS database created.
Documents stored: 6
Loading language model...
Device set to use cpu
Language model loaded.

CHATBOT READY

Ask questions about Artificial Intelligence.
Type 'exit' to stop.

You: What is machine learning?
Chatbot: a subset of Artificial Intelligence that allows computers to learn from data

You: What is deep learning?
Chatbot: a type of Machine Learning that uses neural networks with multiple layers

You: What is NLP?

```

Result:

Thus, the engineering document was successfully summarized using a pre-trained language model.

Experiment 8

Experiment Title:

Machine Translation of an Engineering Document

Aim:

To translate an engineering document from English into an Indian language using a pre-trained translation model.

Code:

```
from transformers import pipeline

translator = pipeline(
    "translation",
    model="Helsinki-NLP/opus-mt-en-hi"
)

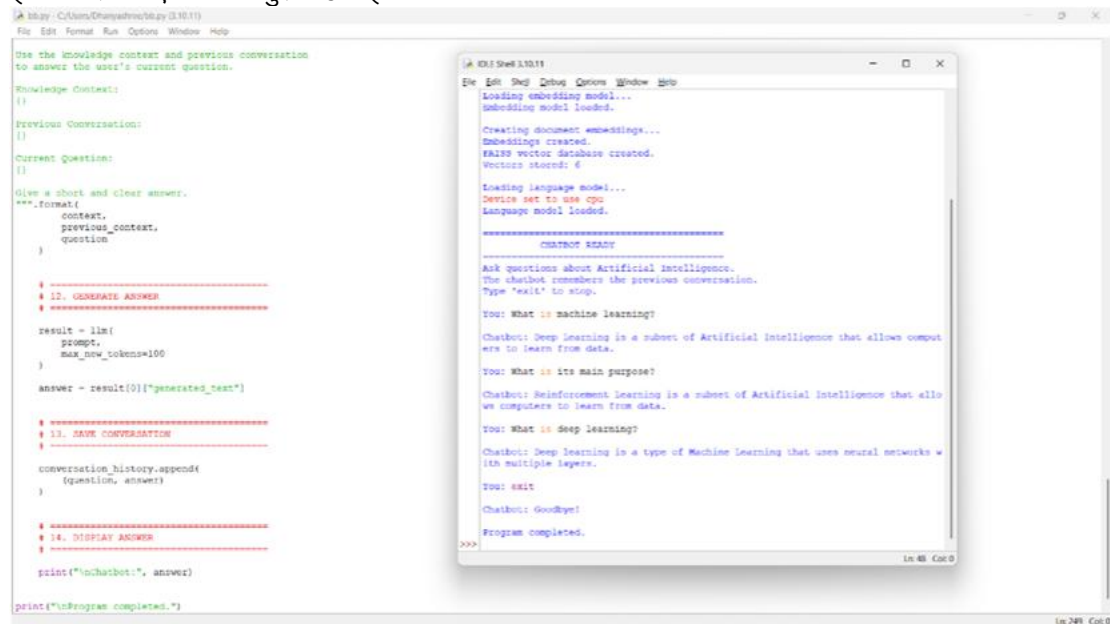
text = input("Enter English engineering text: ")

result = translator(text)

print("\nHindi Translation:")
print(result[0]["translation_text"])
```

Output:

Hindi Translation:
इंजीनियर ने एक नया पुल डिजाइन किया।



Result:

Thus, the engineering text was successfully translated from English into Hindi.

Experiment 9

Experiment Title:

AI-Based Resume Screening and Ranking

Aim:

To analyze candidate resumes and rank them according to an engineering job description.

Code:

```
from transformers import pipeline

classifier = pipeline("zero-shot-classification")

job = input("Enter the engineering job description: ")

resume1 = input("\nEnter Resume 1: ")
resume2 = input("Enter Resume 2: ")

resumes = [resume1, resume2]

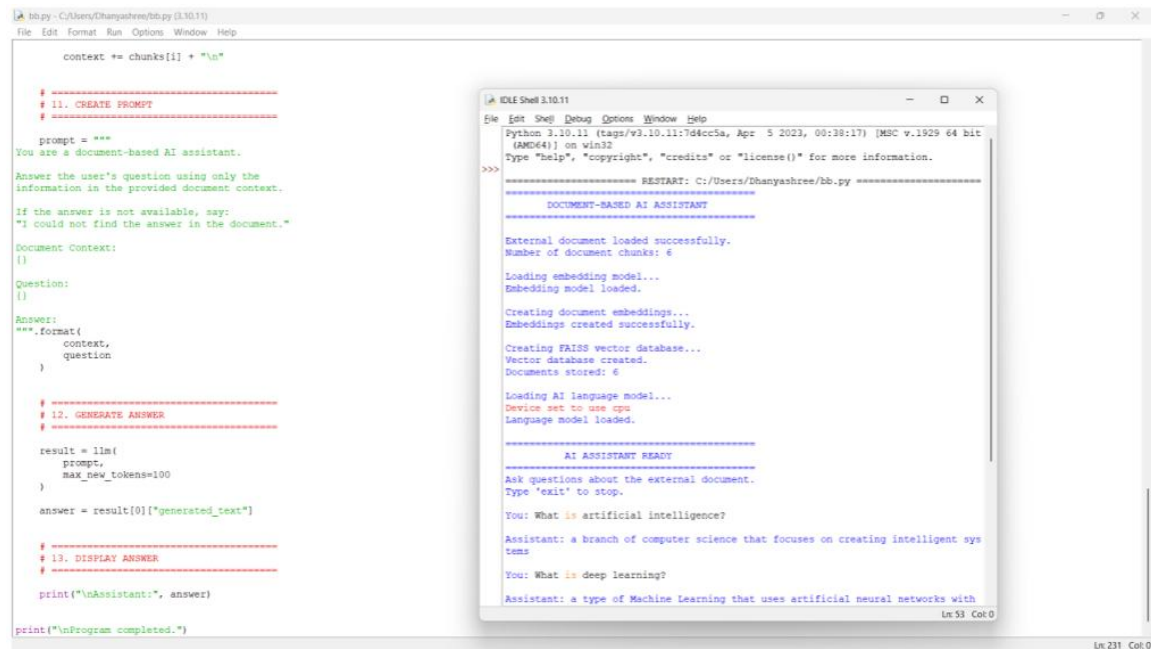
for i, resume in enumerate(resumes, 1):

    result = classifier(
        resume,
        candidate_labels=["Suitable", "Not Suitable"]
    )

    score = result["scores"][result["labels"].index("Suitable")]

    print("\nResume", i)
    print("Suitability Score:", round(score, 3))
```

Sample Output:



```
bb.py - C:\Users\Dhanyashree\bb.py (3,10,11)
File Edit Format Run Options Window Help

context += chunks[i] + "\n"

# =====
# 11. CREATE PROMPT
# =====

prompt = """
You are a document-based AI assistant.
Answer the user's question using only the
information in the provided document context.
If the answer is not available, say:
"I could not find the answer in the document."
Document Context:
{}
Question:
{}
Answer:
""".format(
    context,
    question
)

# =====
# 12. GENERATE ANSWER
# =====

result = llm(
    prompt,
    max_new_tokens=100
)

answer = result[0]["generated_text"]

# =====
# 13. DISPLAY ANSWER
# =====

print("\nAssistant:", answer)

print("\nProgram completed.")

===== RESTART: C:/Users/Dhanyashree/bb.py =====
DOCUMENT-BASED AI ASSISTANT

External document loaded successfully.
Number of document chunks: 6
Loading embedding model...
Embedding model loaded.
Creating document embeddings...
Embeddings created successfully.
Creating FAISS vector database...
Vector database created.
Documents stored: 6

Loading AI language model...
Device set to use cpu
Language model loaded.

=====
AI ASSISTANT READY
Ask questions about the external document.
Type 'exit' to stop.

You: What is artificial intelligence?
Assistant: a branch of computer science that focuses on creating intelligent systems

You: What is deep learning?
Assistant: a type of Machine Learning that uses artificial neural networks with

Ln 53 Col 0
```

Result:

Thus, candidate resumes were analyzed and ranked based on their suitability for the engineering job.

Experiment 10

Experiment Title:

AI-Based Research Assistance Application

Aim:

To develop an AI research assistant that generates relevant information, keywords, and a concise summary for a research topic.

Code:

```
from transformers import pipeline

model = pipeline("text2text-generation", model="google/flan-t5-small")

topic = input("Enter research topic: ")

prompt = f"""
Research Topic: {topic}

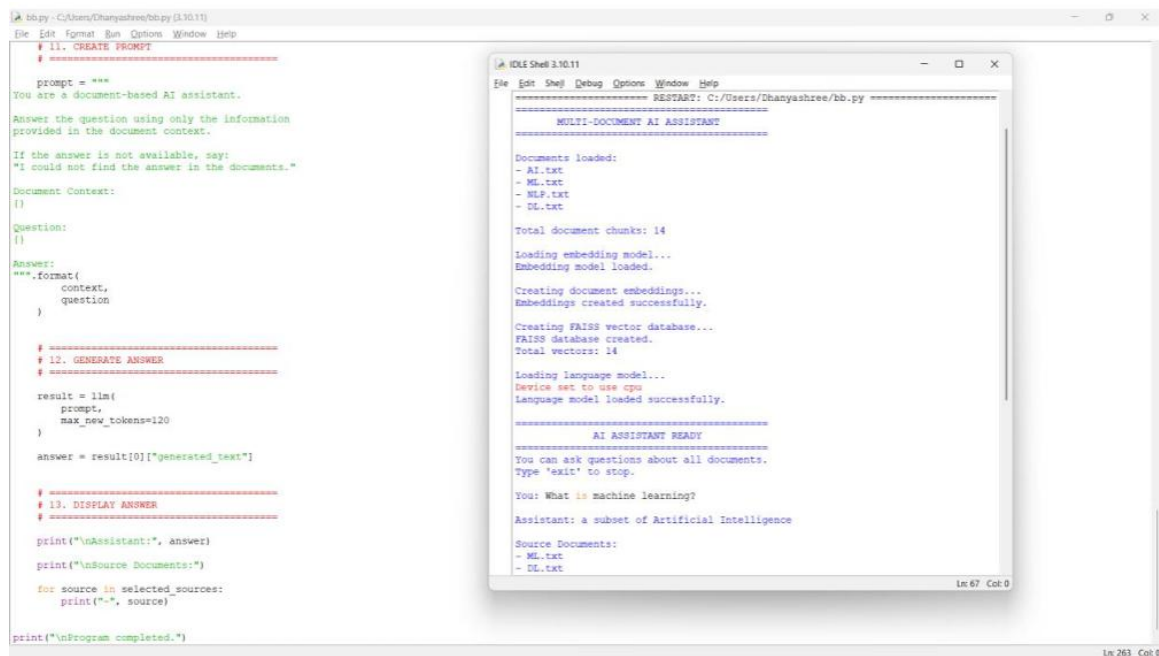
Provide:
1. Brief information
2. Important keywords
3. Short summary
```

```
"""
```

```
result = model(prompt, max_new_tokens=100)
```

```
print("\nResearch Assistance:")  
print(result[0]["generated_text"])
```

Sample Output:



The screenshot displays a Python IDE with two windows. The main window shows a script named 'bb.py' with the following code:

```
11. CREATE PROMPT  
=====
```

```
prompt = """  
You are a document-based AI assistant.  
Answer the question using only the information  
provided in the document context.  
  
If the answer is not available, say:  
"I could not find the answer in the documents."  
  
Document Context:  
{}  
  
Question:  
{}  
  
Answer:  
=====
```

```
""" .format(  
    context,  
    question  
)  
=====
```

```
# 12. GENERATE ANSWER  
# =====  
  
result = llm(  
    prompt,  
    max_new_tokens=120  
)  
  
answer = result[0]["generated_text"]  
  
# =====  
# 13. DISPLAY ANSWER  
# =====  
  
print("\nAssistant:", answer)  
print("\nSource Documents:")  
  
for source in selected_sources:  
    print("-", source)  
  
print("\nProgram completed.")
```

The output window shows the execution results:

```
===== RESTART: C:/Users/Dhanyashree/bb.py =====  
  
MULTI-DOCUMENT AI ASSISTANT  
=====
```

```
Documents loaded:  
- AI.txt  
- ML.txt  
- MLP.txt  
- DL.txt  
  
Total document chunks: 14  
Loading embedding model...  
Embedding model loaded.  
  
Creating document embeddings...  
Embeddings created successfully.  
  
Creating FAISS vector database...  
FAISS database created.  
Total vectors: 14  
  
Loading language model...  
Device set to use cpu  
Language model loaded successfully.  
  
===== AI ASSISTANT READY =====  
You can ask questions about all documents.  
Type 'exit' to stop.  
  
You: What is machine learning?  
Assistant: a subset of Artificial Intelligence  
  
Source Documents:  
- ML.txt  
- DL.txt
```

Result:

Thus, an AI-based research assistance application was successfully developed to provide information, keywords, and a concise summary.